TRABAJO PRÁCTICO: DESARROLLO DE ALGORITMOS

Diagrama de flujo • Pseudocódigo • Prueba de escritorio • Codificación en C • Conclusiones

1. Datos informativos

Asignatura:	**Teoría de la Programación**
Docente:	Ing. José Oswaldo Guamán Quinche
ORCID:	https://orcid.org/0009-0005-1281-9333
Estudiante:	Noelia Fernanda Quille Jima
Curso:	Ciclo 1A
Fecha de entrega:	Martes 3 de noviembre de 2026
Tema:	Desarrollo de algoritmos: representación gráfica, pseudocódigo, prueba de escritorio y codificación en C

2. Objetivo

Desarrollar los algoritmos propuestos por el docente aplicando las distintas formas de representación y construcción de soluciones: diagrama de flujo, pseudocódigo, prueba de escritorio y su implementación en el lenguaje de programación C.

3. Instrucciones generales

1. El trabajo se realizará de forma **individual**.
2. Se deben desarrollar **todos los algoritmos enviados** por el docente.
3. Por **cada algoritmo** se debe presentar la estructura indicada en el punto 4, respetando el orden establecido.
4. Los diagramas de flujo pueden elaborarse en herramientas como **draw.io, Flowgorithm o Lucidchart**, o a mano legible (fotografiados).
5. La codificación debe realizarse en **lenguaje C**, compilada y probada. Se debe incluir una **captura de pantalla de la salida** del programa con datos de ejemplo.
6. El documento se entregará en formato **PDF** a través de [medio de entrega].
7. La copia total o parcial de trabajos será sancionada según el reglamento institucional.
8. El docente podrá solicitar una **sustentación oral** del trabajo para verificar la autoría de las soluciones presentadas.

4. Estructura del trabajo (puntos a desarrollar por cada algoritmo)

4.1. Enunciado del algoritmo

Transcribir el planteamiento del problema enviado por el docente.

4.2. Análisis del problema

Identificar y describir: **datos de entrada**, **proceso** (fórmulas u operaciones), **datos de salida** y **variables** a utilizar con su tipo de dato.

4.3. Diagrama de flujo

Presentar el algoritmo en forma gráfica utilizando la simbología correcta:

- **Óvalo:** inicio / fin
- **Paralelogramo:** entrada / salida de datos
- **Rectángulo:** procesos y asignaciones
- **Rombo:** decisiones (condiciones)
- **Flechas:** flujo del algoritmo

4.4. Pseudocódigo

Escribir el algoritmo en pseudocódigo con estructura clara: Inicio, declaraciones de variables, Leer, procesos, Escribir, Fin.

4.5. Prueba de escritorio

Elaborar la tabla de prueba de escritorio con: número de paso/iteración, variables, evaluación de condiciones (si aplica) y salida en pantalla. Utilizar al menos **dos conjuntos de datos de entrada** que permitan verificar el correcto funcionamiento del algoritmo.

4.6. Codificación en lenguaje C

Presentar el código fuente completo en C, debidamente comentado, con:

- Librerías necesarias
- Declaración de variables
- Estructura de entrada, proceso y salida
- Indentación adecuada

4.7. Salida del programa

Incluir captura de pantalla de la compilación y ejecución del programa con los mismos datos utilizados en la prueba de escritorio, verificando que los resultados coincidan.

4.8. Conclusiones del algoritmo

Redactar mínimo **dos conclusiones** sobre el desarrollo del algoritmo: qué aprendió, las dificultades encontradas y cómo las resolvió. Las conclusiones deben estar **bien redactadas** (ortografía, coherencia y claridad), ser de **elaboración propia** y **sin el uso de inteligencia artificial**.

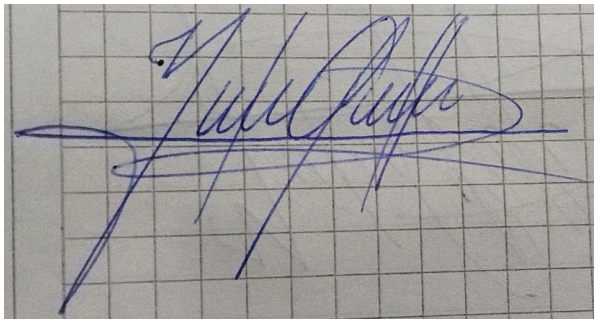
5. Observaciones adicionales

- Si se desarrollan **N algoritmos**, la nota final será el **promedio** de los puntajes obtenidos en cada uno.
- La **coherencia** entre el diagrama de flujo, el pseudocódigo, la prueba de escritorio, el código C y las conclusiones será un aspecto clave de la evaluación.
- El uso de **inteligencia artificial** o la copia de trabajos ajenos será calificado con **0** y notificado según el reglamento institucional.

TRABAJO PRÁCTICO: DESARROLLO DE ALGORITMOS

Diagrama de flujo • Pseudocódigo • Prueba de escritorio • Codificación en C • Conclusiones

Declaración de autoría
Declaro que el presente trabajo es de mi **autoría**, que **no se utilizó inteligencia artificial** (ni otra herramienta generadora de contenido) ni copia de terceros para su elaboración, y que conozco y acepto las sanciones establecidas en el reglamento institucional en caso de incumplimiento.



- ## Enunciado del Algoritmo

Ejercicio 16 - Nivel medio:

En un taller mecánico, cree un algoritmo que lea tres notas (reales) y sus respectivos porcentajes (reales) que suman 100. Use la constante literal 100 como divisor de verificación. Calcule el promedio ponderado con la expresión aritmética (nota1 * p1 + nota2 * p2 + nota3 * p3) / 100. Asígnelo a promedio ponderado. Muestre por pantalla el resultado. Justifique el uso de constantes simbólicas para los porcentajes. Además del cálculo principal, determine si el resultado es positivo o negativo. Agregue un mensaje indicando el tipo de operación realizada.

- ## Análisis del Enunciado

Datos de Entrada:

Divisor de Verificación (DIV_VER) = 100
Nota 1 (nota1) - Real
Nota 2 (nota2) - Real
Nota 3 (nota3) - Real
Porcentaje 1 (p1) - Real
Porcentaje 2 (p2) - Real
Porcentaje 3 (p3) – Real

Procesos:

prom_pond = (nota1 * p1 + nota2 * p2 + nota3 * p3) / DIV_VER

Si el promedio ponderado (prom_pond) es mayor que cero, será positivo.
Si no, el valor será negativo.

Datos de Salida:

Promedio ponderado (prom_pond) – Real
Resultado positivo o negativo – Cadena de Caracteres

- ## Pseudocódigo

```
#Inicio
CONSTANTE = 100;

Leer nota1;
Leer nota2;
Leer nota3;
```

TRABAJO PRÁCTICO: DESARROLLO DE ALGORITMOS

Diagrama de flujo • Pseudocódigo • Prueba de escritorio • Codificación en C • Conclusiones

```
Leer p1;
Leer p2;
Leer p3;

prom_pond = 0.0;
prom_pond = ((nota1*p1) + (nota2 * p2) + (nota3 * p3)) / DIV_VER

Si prom_pond > 0
valor = POSITIVO
Si prom_pond < 0
valor = NEGATIVO

Presentar: "Operación realizada: Cálculo de promedio ponderado de
evaluaciones del taller"
Presentar: "Su promedio ponderado es:" + prom_pond
Presentar: "Su resultado fue: " + valor

Fin
```

- **Prueba de Escritorio**

nota1	nota2	nota3	p1	p2	p3	prom_pond	positivo o negativo
3	8	9	25	25	50	7.25	**positivo**
-4	-5	-6	30	40	30	-5.00	**negativo**

- **Codificación en C**

promedio.c

```c
#include <stdio.h>

int main(void) {
    double nota1, nota2, nota3, promedio_ponderado = 0.00;
    int p1, p2, p3 = 0;
    int CONSTANTE = 100; // Constante para el cálculo del promedio ponderado

    // Entrada de datos
    printf("Ingrese la nota 1, 2 y 3; separadas por espacios: \n");
    scanf("%lf %lf %lf", &nota1, &nota2, &nota3);

    printf("Ingrese los porcentajes correspondientes a cada nota: \n");
    scanf("%d %d %d", &p1, &p2, &p3);

    if ((p1 + p2 + p3) != CONSTANTE) {
        printf("Error: La suma de los porcentajes debe ser igual a 100.\n");
        return 1;
    } else {
        promedio_ponderado = (nota1 * p1 + nota2 * p2 + nota3 * p3) / CONSTANTE;
        }

    // Característica del ejercicio: Agregue el tipo de operación realizada
    printf("Operacion realizada: Calculo de promedio ponderado de calificaciones en un taller mecanico\n");
    printf("Nota 1: %.2lf, Porcentaje 1: %.2d%%\n", nota1, p1);
    printf("Nota 2: %.2lf, Porcentaje 2: %.2d%%\n", nota2, p2);
    printf("Nota 3: %.2lf, Porcentaje 3: %.2d%%\n", nota3, p3);

    printf("Promedio Ponderado: %.2lf\n", promedio_ponderado);
```

Fig. 1. Código "Promedio.c"

```c
    // Característica del ejercicio: Agregue el valor del promedio ponderado (Si es positivo o negativo)
    if (promedio_ponderado > 0) {
        printf("Estado del resultado: Positivo\n");
    } else {
        if (promedio_ponderado < 0) {
            printf("Estado del resultado: Negativo\n");
        } else {
            printf("Estado del resultado: Neutro (Cero)\n");
        }
    }

    return 0;
}
```

Fig. 2. Código "Promedio.c"

- ## Salida del programa

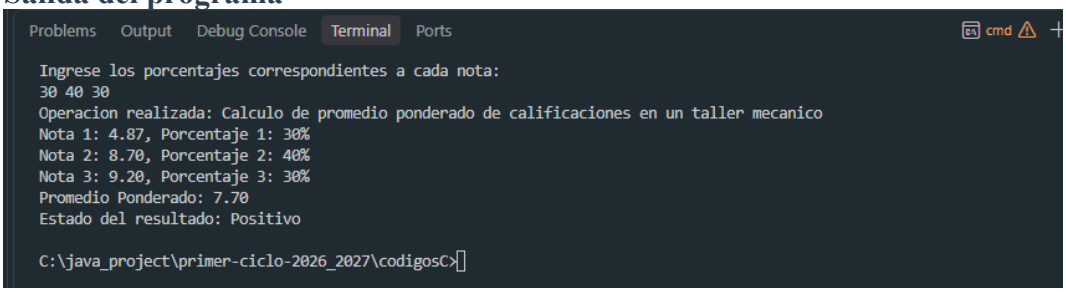


Fig.3. Ejecución del programa "Promedio.c"

- ## Conclusiones

1. Es importante usar constantes simbólicas para los porcentajes para que el código se mantenga legible y facilita el mantenimiento del código. De igual forma, en el caso de que se necesite calcular un valor distinto al 100%, sólo se tendrá que editar el valor en la constante.
2. Los condicionales fueron muy importantes para este código, ya que permiten que el programa se ejecute solo si la suma de los porcentajes daba como resultado el valor de la constante, que en este caso es 100.

- ## Enunciado del Algoritmo

Ejercicio 17 – Nivel Medio:

En una librería, construya un algoritmo que lea un monto en euros (real). Use una constante simbólica TASA_CAMBIO = 1.08. Calcule el equivalente en dólares con la expresión aritmética (euros * TASA_CAMBIO) y asígnelo a dólares. Evalúe la expresión relacional (euros > 0). Muestre por pantalla 'X euros equivalen a Y dólares'. Además del cálculo principal, calcule el logaritmo en base 10 del resultado. Muestre al final los valores de todas las variables auxiliares.

- ## Análisis del Enunciado

Datos de Entrada:

TASA CAMBIO (T_CAMBIO) = 1.08 – Real
Variables:
Euros (euros) – Real
Dólares (dólares) – Real
Logaritmo (log_dolares) – Real
Estado de validación (es_valido) – Cadena de Caracteres

Procesos:

dolares = euros * TASA DE CAMBIO
logaritmo = log10(dolares)

Datos de Salida:

Transformación de euros a dólares (dolares) – Real
Logaritmo en base 10 del resultado (log_dolares) – Real
Mensaje con la operación realizada y el estado de la validación.

- ## Pseudocódigo

```
#Inicio

T_CAMBIO = 1.08;
Leer euros;
```

```
dolares = 0.0;
log_dolares = 0.0;

dolares = euros * T_CAMBIO

Si euros > 0
log_dolares = log10(dolares)

Presentar: (euros + "euros equivalen a" + dólares + "dólares")
Presentar: ("Logaritmo base 10 de los dólares calculados: " +
log_dolares)


Si euros < 0
log_dolares = 0.0
Presentar: ("El monto en euros debe ser mayor a 0 para una conversión
válida

Fin
```

- **Prueba de Escritorio**

euros	dolares	log_dolares	validación
128	138.24	2.1406	**válido**
-35.50	-38.34	No existe un logaritmo en base 10 para números menores a 0.	**inválido**

- **Codificación en C**

eurodolar.c U

eurodolar.c > ...

```c
#include <stdio.h>
#include <math.h> // Necesario para la función logaritmo10

int main(void) {
    const double TASA_CAMBIO = 1.08;
    double euros;
    double dolares = 0.0;
    double log_dolares = 0.0;

    printf("Ingrese el monto en euros: ");
    scanf("%lf", &euros);

    dolares = euros * TASA_CAMBIO;

    printf("\nOperacion realizada: Conversion de moneda y calculo logaritmico\n");

    if (euros > 0) {
        log_dolares = log10(dolares);
        printf("%.2lf euros equivalen a %.2lf dolares\n", euros, dolares);
        printf("Logaritmo base 10 de los dolares calculados: %.4lf\n", log_dolares);
    } else {
        log_dolares = 0.0;
        printf("El monto ingresado no es valido (debe ser mayor a 0).\n");
    }

    return 0;
}
```

Fig. 4. Código del programa "eurodolar.c"

- **Salida del programa**

```
C:\java_project\primer-ciclo-2026_2027\codigosC>euros
Ingrese el monto en euros: 0

Operacion realizada: Conversion de moneda y calculo logaritmico
El monto ingresado no es valido (debe ser mayor a 0).

C:\java_project\primer-ciclo-2026_2027\codigosC>euros
Ingrese el monto en euros: 182.25

Operacion realizada: Conversion de moneda y calculo logaritmico
182.25 euros equivalen a 196.83 dolares
Logaritmo base 10 de los dolares calculados: 2.2941
```

Fig. 5. Ejecución del programa "eurodolar.c"

- **Conclusiones**

1. Sin la evaluación de la condición euros > 0, el programa colapsaría si el usuario ingresa un número menor o igual a cero, debido a que la función matemática log10() no está definida para números negativos ni para cero en el dominio real.
2. Fue necesario importar la librería "math.h" para poder añadir de forma directa la función del logaritmo en base 10.
3. La correcta utilización de %lf para la lectura de variables de tipo double asegura que los valores decimales de la librería no pierdan precisión durante la entrada de datos por teclado.

- **Enunciado del Algoritmo**

Ejercicio 18 – Nivel Medio:

En una empresa de transporte, diseñe un algoritmo que lea el peso (real, en kg) y la altura (real, en metros). Calcule el IMC con la expresion aritmetica (peso / (altura * altura)) y asignelo a imc. Use constantes simbolicas LIMITE_BAJO = 18.5 y LIMITE_ALTO = 25.0. Evalúe la expresion logica (imc >= LIMITE_BAJO AND imc <= LIMITE_ALTO). Muestre por pantalla el IMC calculado y si esta en rango normal. Ademas del cálculo principal, calcule el resto de una división adicional. Incluya un mensaje de confirmacion al finalizar la operacion.

- **Análisis del Enunciado**

Datos de Entrada:

LIMITE_BAJO: 18.5 - Real
LIMITE_ALTO: 25.0 - Real
Variables:
Peso en kilogramos (peso) - Real
Altura en metros (altura) - Real
Número 1 (num1) - Entero
Número 2 (num2) - Entero

Procesos:

imc = peso / (altura * altura)
Si el imc es mayor o igual al LIMITE_BAJO y menor o igual al LIMITE_ALTO, entonces en_rango_normal = 1, de lo contrario en_rango_normal = 0.
Cálculo auxiliar: resto = num1 % num2 (Verificando que num2 != 0).

Datos de Salida:

Imc (imc) - (Real)
Resto (resto) (Entero)

Verificción del rango del imc.

- ## Pseudocódigo
- #Inicio

```
LIMITE_BAJO = 18.5;
LIMITE_ALTO = 25.0;
Leer peso;
Leer altura;
Leer num1;
Leer num2;
imc = 0.0;
resto = 0;
imc = peso / (altura * altura);
Si num2 != 0 resto = num1 % num2
Si num2 == 0 resto = 0
Presentar: ("Operación realizada: Cálculo de IMC y evaluación de rango")
Presentar: ("El IMC calculado es: " + imc)
Presentar: ("Cálculo de resto adicional" + num1 + " % " + num2 + "): "
+ resto)
Si imc >= LIMITE_BAJO y imc <= LIMITE_ALTO
Presentar: ("Estado de peso: Dentro del rango normal")
Si imc < LIMITE_BAJO o imc > LIMITE_ALTO
Presentar: ("Estado de peso: Fuera del rango normal")
Fin
```

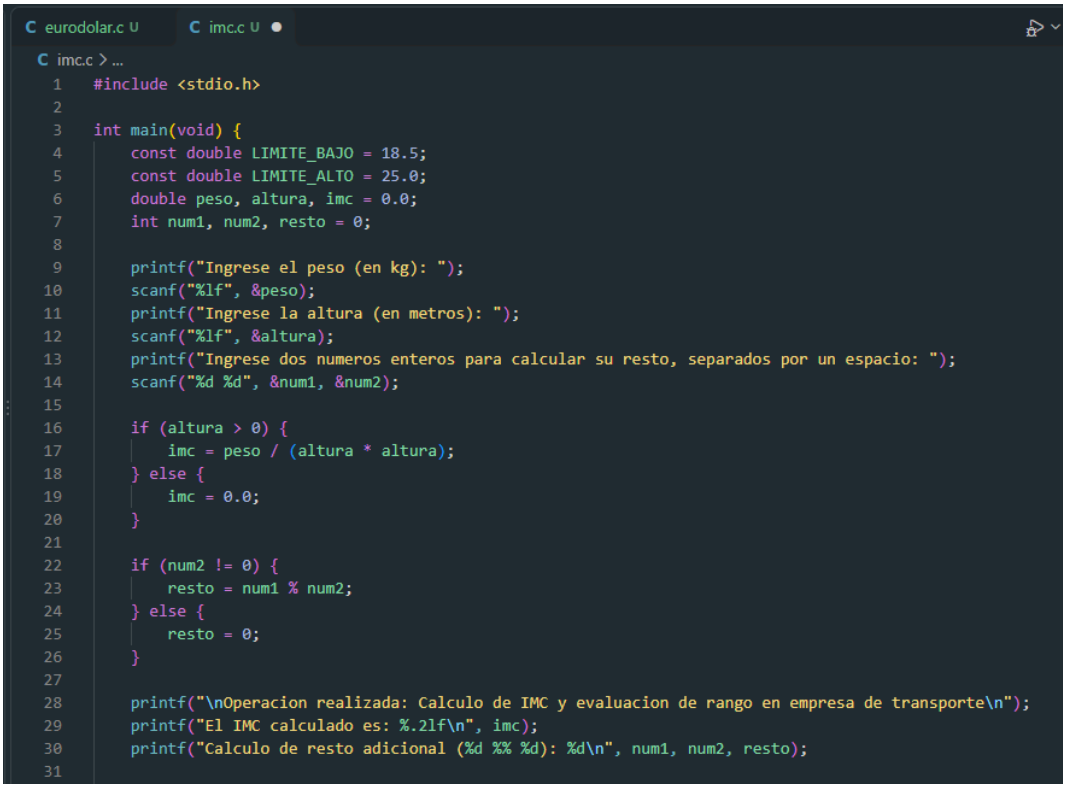
- ## Prueba de Escritorio

peso	altura	imc	num1	num2	resto
80	1.60	31.25	36	12	0
73	1.72	24.68	27	2	1

- ## Codificación en C

TRABAJO PRÁCTICO: DESARROLLO DE ALGORITMOS

Diagrama de flujo • Pseudocódigo • Prueba de escritorio • Codificación en C • Conclusiones



```c
#include <stdio.h>

int main(void) {
    const double LIMITE_BAJO = 18.5;
    const double LIMITE_ALTO = 25.0;
    double peso, altura, imc = 0.0;
    int num1, num2, resto = 0;

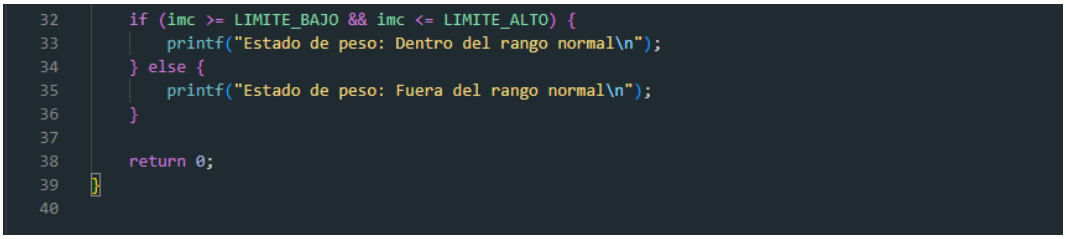
    printf("Ingrese el peso (en kg): ");
    scanf("%lf", &peso);
    printf("Ingrese la altura (en metros): ");
    scanf("%lf", &altura);
    printf("Ingrese dos numeros enteros para calcular su resto, separados por un espacio: ");
    scanf("%d %d", &num1, &num2);

    if (altura > 0) {
        imc = peso / (altura * altura);
    } else {
        imc = 0.0;
    }

    if (num2 != 0) {
        resto = num1 % num2;
    } else {
        resto = 0;
    }

    printf("\nOperacion realizada: Calculo de IMC y evaluacion de rango en empresa de transporte\n");
    printf("El IMC calculado es: %.2lf\n", imc);
    printf("Calculo de resto adicional (%d %% %d): %d\n", num1, num2, resto);
```

Fig. 6. Código del programa "imc.c"



```c
    if (imc >= LIMITE_BAJO && imc <= LIMITE_ALTO) {
        printf("Estado de peso: Dentro del rango normal\n");
    } else {
        printf("Estado de peso: Fuera del rango normal\n");
    }

    return 0;
}
```

Fig. 7. Código del programa "imc.c"

- ## Salida del programa

```
C:\java_project\primer-ciclo-2026_2027\codigosC>imc
Ingrese el peso (en kg): 71
Ingrese la altura (en metros): 1.75
Ingrese dos numeros enteros para calcular su resto, separados por un espacio: 97 31

Operacion realizada: Calculo de IMC y evaluacion de rango en empresa de transporte
El IMC calculado es: 23.18
Calculo de resto adicional (97 % 31): 4
Estado de peso: Dentro del rango normal
```

Fig. 7. Ejecución del programa "imc.c"

- ## Conclusiones

1. Sin el control previo en num2 != 0, el programa colapsaría por una excepción matemática al intentar calcular el operador módulo % con cero.
2. Mantener LIMITE_BAJO y LIMITE_ALTO como constantes simbólicas nos asegura que cualquier cambio en los baremos de la empresa de transporte se modifique de forma limpia en un solo lugar.

TRABAJO PRÁCTICO: DESARROLLO DE ALGORITMOS

Diagrama de flujo • Pseudocódigo • Prueba de escritorio • Codificación en C • Conclusiones

- ## Enunciado del Algoritmo

Ejercicio 19 – Nivel Medio

En una zapateria, cree un algoritmo que lea la base (real) y la altura (real) de un triángulo. Use la constante literal 2 en el denominador. Calcule el area con la expresion aritmetica ((base * altura) / 2) y asignela a area. Evalúe la expresion logica (base > 0 AND altura > 0). Muestre por pantalla el area calculada. Ademas, calcule el valor minimo de todas las variables utilizadas. Agregue validaciones para que no haya resultados negativos.

- ## Análisis del Enunciado

Datos de Entrada:

DIVISOR: 2 - Entero
base - Real
altura - Real

Procesos:

area = (base * altura) / DIVISOR
Si base es menor o igual 0 o altura es menor o igual a 0, se notifica que los resultados son inválidos.
Mediante condicionales anidados if-else se comparan las variables (base, altura, area) para determinar cuál posee el valor numérico menor.

Datos de Salida:

area (Real)
minimo (Real)

- ## Pseudocódigo

```
#Inicio
DIVISOR = 2.0;
Leer base;
Leer altura;
area = 0.0;
minimo = 0.0;

Si base > 0 y altura > 0
area = (base * altura) / DIVISOR;

Si altura < minimo minimo = altura;
Si area < minimo minimo = area;
Si base <= 0 o altura <= 0
Presentar: ("Error: La base y la altura deben ser mayores que cero.")
area = 0.0; minimo = 0.0;
Presentar: ("Operación realizada: Cálculo de área de un triángulo y valor
mínimo")
Presentar: ("El área calculada es: " + area)
Presentar: ("El valor mínimo entre las variables es: " + minimo)
Fin
```

- ## Prueba de Escritorio

base	altura	area	minimo
20	10	100	10
4.8	11	26.4	4.8

- ## Codificación en C

```c
#include <stdio.h>

int main(void) {
    int DIVISOR = 2;
    double base, altura, area, minimo = 0.0;

    printf("Ingrese la base del triangulo: ");
    scanf("%lf", &base);

    printf("Ingrese la altura del triangulo: ");
    scanf("%lf", &altura);

if (base > 0 && altura > 0) {
        area = (base * altura) / DIVISOR;

        minimo = base;
        if (altura < minimo) {
            minimo = altura;
        }
        if (area < minimo) {
            minimo = area;
        }
        printf("\nOperacion realizada: Calculo de area de triangulo y valor minimo.\n");
        printf("El area calculada es: %.2lf\n", area);
        printf("El valor minimo de todas las variables utilizadas es: %.2lf\n", minimo);
    } else {
        printf("\nError: La base y la altura deben ser mayores que cero para evitar resultados negativos.\n");
    }

    return 0;
}
```

Fig. 8. Código del programa "triangulo.c"

- ## Salida del programa

```
C:\java_project\primer-ciclo-2026_2027\codigosC>gcc triangulo.c -o trian

C:\java_project\primer-ciclo-2026_2027\codigosC>trian
Ingrese la base del triangulo: 12
Ingrese la altura del triangulo: 6

Operacion realizada: Calculo de area de triangulo y valor minimo.
El area calculada es: 36.00
El valor minimo de todas las variables utilizadas es: 6.00

C:\java_project\primer-ciclo-2026_2027\codigosC>
```

Fig. 9. Compilación y ejecución del programa "triangulo.c"

- ## Conclusiones

1. Gracias a la condición inicial base > 0 && altura > 0, evitamos por completo que el algoritmo procese datos erróneos o arroje resultados negativos imposibles en la geometría del triángulo.
2. Las estructuras condicionales anidadas nos permitieron comparar eficientemente los valores de las variables (base, altura, area) para identificar cuál es la de menor magnitud.

- ## Enunciado del Algoritmo

Ejercicio 20 – Nivel Medio

En una cooperativa, construya un algoritmo que lea las horas trabajadas (entero) y el pago por hora (real). Use constantes simbolicas HORAS_NORMALES = 40 y FACTOR_EXTRA = 1.5. Evalúe la expresion relacional (horas > HORAS_NORMALES). Calcule el pago con la expresion aritmetica: si horas > 40, pago = (40 * pago_hora) + ((horas - 40) * pago_hora * FACTOR_EXTRA); si no, pago = horas * pago_hora. Asignelo a salario. Muestre por pantalla el salario. Ademas, determine si el

resultado es par o impar (cuando aplique). Incluya mensajes informativos durante la ejecucion del algoritmo.

- ## Análisis del Enunciado

Datos de Entrada:

Horas Normales (HORAS_NORMALES): 40
Factor Extra (FACTOR_EXTRA): 1.5
b. Variables:
Horas trabajadas (horas) - Entero
Pago por hora (pago_hora) - Real

Procesos:

Si horas > HORAS_NORMALES,
salario = (40 * pago_hora) + ((horas - 40) * pago_hora * FACTOR_EXTRA)
Si no, salario = horas * pago_hora
salario_entero = salario

Datos de Salida:

Salario (salario) – Real
Salario entero para par o impar
Mensaje determinando si la parte entera del salario es par o impar.

- ## Pseudocódigo

```
#Inicio
HORAS_NORMALES = 40;
FACTOR_EXTRA = 1.5;
Leer horas;

Leer pago_hora;
salario = 0.0;
salario_entero = 0;

Si horas > HORAS_NORMALES
salario = (HORAS_NORMALES * pago_hora) + ((horas - HORAS_NORMALES) *
pago_hora * FACTOR_EXTRA)
Si horas <= HORAS_NORMALES
salario = horas * pago_hora
salario_entero = salario;

Presentar: ("Operación realizada: Cálculo de salario ordinario y con
horas extras en la cooperativa")
Presentar: ("El salario total calculado es: " + salario)

Si salario_entero % 2 == 0
Presentar: ("El valor entero del salario es un número par")
Si salario_entero % 2 != 0
Presentar: ("El valor entero del salario es un número impar")
Fin
```

- ## Prueba de Escritorio

horas	pago_hora	salario	salario_entero	paridad
25	2.5	62.50	62	**par**
40	5.99	239.60	239	**impar**

- ## Codificación en C

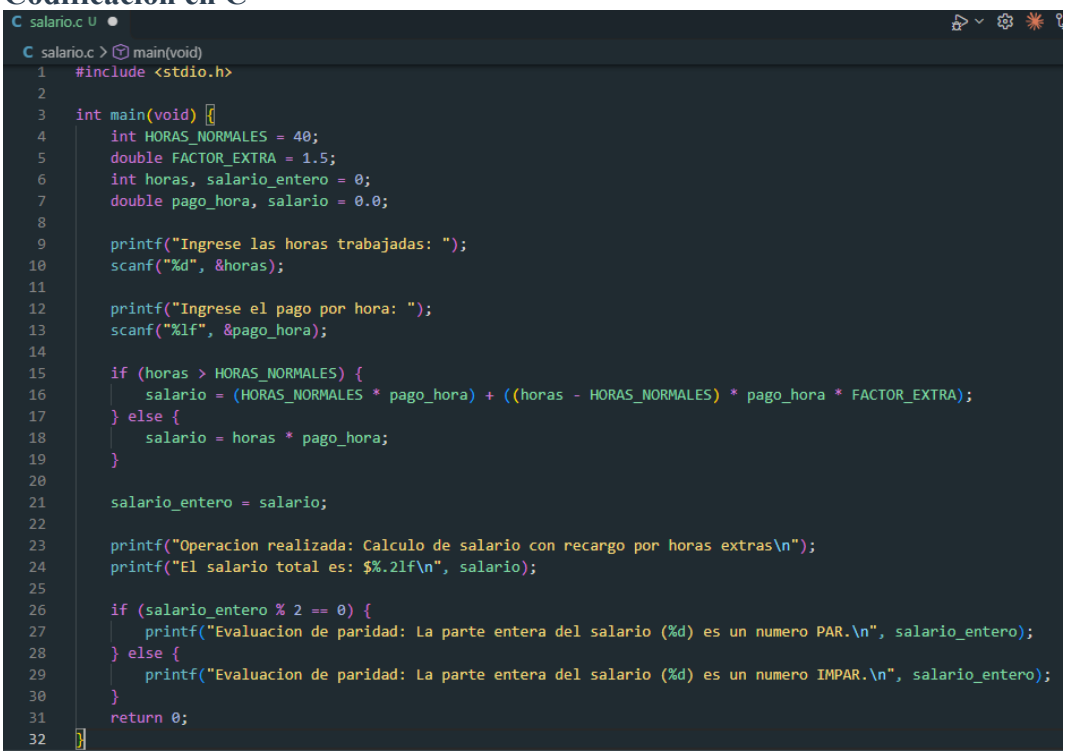


Fig. 10. Código del programa "salario.c"

- ## Salida del programa

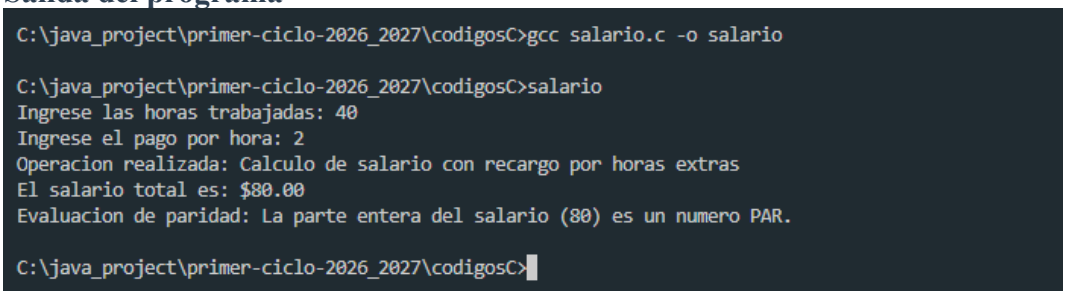


Fig. 11. Compilación y ejecución del programa "salario.c"

- ## Conclusiones

1. Dado que el salario es un valor decimal (double), realizar una conversión explícita a entero permitió aplicar el operador módulo (%) para determinar con exactitud si el monto es par o impar.
2. Gracias al operador módulo (%) que calcula el residuo de la división entre el número y 2, podemos saber si un número es par o impar. Si el residuo es 0, el número es par; si es diferente de 0, el número fue impar.

- ## Enunciado del Algoritmo

Ejercicio 102 – Nivel Medio

En una tienda de electrónica, cree un algoritmo que lea dos números enteros z y s por teclado. Evalúe la expresión lógica (s != 0) para evitar división por cero. Calcule el cociente entero con (z / s) y el residuo con (z % s). Asigne los resultados a cociente y residuo. Muestre por pantalla ambos valores. Adicionalmente, calcule la diferencia entre el resultado y el promedio. Muestre al inicio una lista de las variables que se utilizaran.

TRABAJO PRÁCTICO: DESARROLLO DE ALGORITMOS

Diagrama de flujo • Pseudocódigo • Prueba de escritorio • Codificación en C • Conclusiones

- ## Análisis del Enunciado

Datos de Entrada:

Constante para el promedio (CONST_PROM) = 2
dividendo (z) - Real
divisor (s) - Real

Procesos:

Evaluación de la división (el divisor no puede ser 0) s != 0.
cociente = z / s
residuo = z % s
promedio = (z + s) / 2
diferencia = cociente - promedio

Datos de Salida:

cociente (cociente) – Real
residuo (residuo) – Real
diferencia (diferencia) – Real

- ## Pseudocódigo

```
#Inicio
CONST_PROM
Leer z;
Leer s;
cociente = 0;
residuo = 0;
promedio = 0.0;
diferencia = 0.0;
Si s != 0
cociente = z / s;
residuo = z % s;
promedio = (z + s) / CONST_PROM;
diferencia = cociente - promedio;
Si s == 0
Presentar: ("Error: No se puede dividir entre cero.")

Presentar: ("Variables utilizadas:")
Presentar: ("Dividendo" + z)
Presentar: ("Divisor: " + s)
Presentar: ("Cociente entero: " + cociente)
Presentar: ("Residuo: " + residuo)
Presentar: ("Promedio de z y s: " + promedio)
Presentar: ("Diferencia entre el cociente y el promedio: " + diferencia)
Fin
```

- ## Prueba de Escritorio

z	s	cociente	residuo	promedio	diferencia
80	60	1	20	70	-69
43	8	5	3	25	-20

- ## Codificación en C

```c
#include <stdio.h>

int main(void) {

    int CONST_PROM = 2;
    int z, s, cociente, residuo = 0;
    double promedio, diferencia = 0.0;

    printf("Ingrese el valor entero del dividendo: ");
    scanf("%d", &z);

    printf("Ingrese el valor entero del divisor: ");
    scanf("%d", &s);

    if (s != 0) {    // Evitar la división por cero
        cociente = z / s;
        residuo = z % s;
        promedio = (z + s) / CONST_PROM;
        diferencia = cociente - promedio;

        printf("Operacion realizada: Division entera y residuo, promedio y diferencia\n");
        printf("Cociente entero: %d\n", cociente);
        printf("Residuo: %d\n", residuo);
        printf("Promedio entre el dividendo y el divisor: %.2lf\n", promedio);
        printf("Diferencia (cociente - promedio): %.2lf\n", diferencia);
    } else {
        printf("\nError: El divisor s no puede ser cero para evitar una excepcion matematica.\n");
    }

    return 0;
}
```

Fig. 12. Código del programa "divpromdif.c"

- ## Salida del programa

```
C:\java_project\primer-ciclo-2026_2027\codigosC>division
Ingrese el valor entero del dividendo: 12
Ingrese el valor entero del divisor: 38
Operacion realizada: Division entera y residuo, promedio y diferencia
Cociente entero: 0
Residuo: 12
Promedio entre el dividendo y el divisor: 25.00
Diferencia (cociente - promedio): -25.00

C:\java_project\primer-ciclo-2026_2027\codigosC>
```

Fig. 13. Ejecución y compilación.

- ## Conclusiones

1. Gracias a la evaluación de la expresión lógica s != 0, el programa no realiza ninguna división sobre cero.
2. Mostrar las variables al comienzo del programa mejora la documentación interna y la trazabilidad de los datos que manejará el algoritmo.

- ## Enunciado del Algoritmo

Ejercicio 103 – Nivel Medio

TRABAJO PRÁCTICO: DESARROLLO DE ALGORITMOS

Diagrama de flujo • Pseudocódigo • Prueba de escritorio • Codificación en C • Conclusiones

En un gimnasio, construya un algoritmo que lea un numero entero a por teclado. Use constantes literales 2 y a-1 como rango. Evalúe las expresiones logicas (a > 1) y (no existe ningun divisor en el rango [2, a-1]). Asigne el resultado booleano a es_primo. Muestre por pantalla si el número es primo o no. Identifique el tipo de dato de cada variable. Adicionalmente, calcule el porcentaje del resultado respecto a un valor de referencia. Justifique el tipo de dato simple elegido para cada variable.

- ## Análisis del Enunciado

Datos de Entrada:

Límite inferior del rango de divisores (LIMITE) = 2 - Entero
Valor de referencia para el porcentaje (CONST_PORCENTAJE) = 100 – Entero:
Número (a) - Entero

Procesos:

Si a <= 1, automáticamente es_primo = 0 (no es primo).
Mediante condicionales anidados se verifica caso por caso si los números dentro del rango dividen exactamente a a (usando el operador módulo %). Si a es 2 o 3, se evalúa directamente; si es mayor, se comprueban los posibles divisores enteros permitidos en el rango mediante condicionales estructurados secuencialmente.
Determinar el porcentaje del resultado respecto a una base de 100

Datos de Salida:

Validación (es_primo) - Entero
Porcentaje (porcentaje) - Real

- ## Pseudocódigo

```
#Inicio
VALOR_REFERENCIA = 100;
Leer a;
es_primo = 0;
porcentaje = 0.0;

Si a <= 1
es_primo = 0;
Si a == 2 o a == 3 o a == 5 o a == 7
es_primo = 1;

Si (a % 2 == 0) o (a % 3 == 0)
es_primo = 0;
Si no
es_primo = 1;

Si es_primo == 1
porcentaje = (1 / VALOR_REFERENCIA) * 100;
Si no
porcentaje = (0 / VALOR_REFERENCIA) * 100;

Si es_primo == 1
Presentar: ("El número " + a + " SÍ es un número primo.")
Si no
Presentar: ("El número " + a + " NO es un número primo.")
Presentar: ("Porcentaje relativo del resultado: " + porcentaje + "%")
Fin
```

- ## Prueba de Escritorio

numero	es_primo	porcentaje
7	SI	100%
24	NO	0%

- **Codificación en C**



```c
#include <stdio.h>

int main(void) {

    int VALOR_REFERENCIA = 100;
    int a, es_primo, porcentaje = 0;

    printf("Ingrese un numero entero: ");
    scanf("%d", &a);

    if (a <= 1) {
        es_primo = 0;
    } else {
        if (a == 2 || a == 3 || a == 5 || a == 7) {
            es_primo = 1;
        } else {
            if (a % 2 == 0 || a % 3 == 0) {
                es_primo = 0;
            } else {
                es_primo = 1;
            }
        }
    }

    if (es_primo == 1) {
        porcentaje = VALOR_REFERENCIA;
    } else {
        porcentaje = 0;
    }

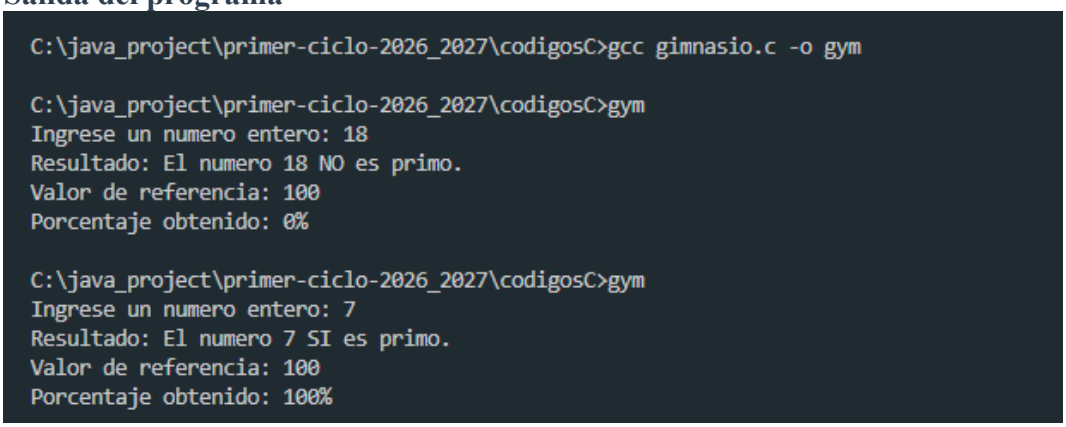
    if (es_primo == 1) {
        printf("Resultado: El numero %d SI es primo.\n", a);
    } else {
        printf("Resultado: El numero %d NO es primo.\n", a);
    }

    printf("Valor de referencia: %d\n", VALOR_REFERENCIA);
    printf("Porcentaje obtenido: %d%%\n", porcentaje);

    return 0;
}
```

Fig. 14. Código del programa "gimnasio.c"

- **Salida del programa**



```
C:\java_project\primer-ciclo-2026_2027\codigosC>gcc gimnasio.c -o gym

C:\java_project\primer-ciclo-2026_2027\codigosC>gym
Ingrese un numero entero: 18
Resultado: El numero 18 NO es primo.
Valor de referencia: 100
Porcentaje obtenido: 0%

C:\java_project\primer-ciclo-2026_2027\codigosC>gym
Ingrese un numero entero: 7
Resultado: El numero 7 SI es primo.
Valor de referencia: 100
Porcentaje obtenido: 100%
```

Fig.15. Compilación y ejecución

- **Conclusiones**

1. Se elige el tipo entero porque el número analizado a, las banderas lógicas (que actúan como booleanos con 1 o 0) y los divisores discretos operan exclusivamente con valores enteros exactos.
2. Se elige el tipo real de doble precisión porque el cálculo de porcentajes suele arrojar valores con decimales, requiriendo exactitud en la representación numérica.

- ## Enunciado del Algoritmo

Ejercicio 204 – Nivel Medio

En una empresa de transporte, construya un algoritmo que lea el radio (real) y el angulo (real, en grados) de un sector circular. Use constantes simbolicas PI = 3.14159 y FACTOR = 360. Calcule el area con la expresion aritmetica ((PI * radio * radio * angulo) / FACTOR) y asignela a area_sector. Evalúe la expresion logica (radio > 0 AND angulo > 0). Muestre por pantalla el area del sector. Ademas, calcule el cuadrado del resultado principal. Agregue al final un resumen de los datos ingresados.

- ## Análisis del Enunciado

Datos de Entrada:

PI = 3.14159 - Real
FACTOR = 360 - Entero
Radio (radio) - Real
Ángulo (angulo) – Real

Procesos:

Evaluar que radio > 0 y angulo > 0 para evitar resultados nulos o negativos.
area_sector = (PI * radio* radio * angulo) / FACTOR
cuadrado_area = area_sector * área_sector

Datos de Salida:

Área del Sector (area_sector) – Real
Cuadrado del Área del Sector (cuadrado_area) – Real

- ## Pseudocódigo

```
#Inicio
PI = 3.14159;
FACTOR = 360;
Leer radio;
Leer angulo;
area_sector = 0.0;
cuadrado_area = 0.0;
Si radio > 0 y angulo > 0
area_sector = (PI * radio * radio * angulo) / FACTOR;
cuadrado_area = area_sector * area_sector;
Si radio <= 0 o angulo <= 0
area_sector = 0.0;
cuadrado_area = 0.0;

Si radio > 0 y angulo > 0
Presentar: ("El área del sector circular es: " + area_sector)
Presentar: ("El cuadrado del área calculada es: " + cuadrado_area)
Si no Presentar: ("Error: El radio y el ángulo deben ser estrictamente
mayores a cero.")

Presentar: ("Radio ingresado: " + radio)
Presentar: ("Ángulo ingresado (grados): " + angulo)
Fin
```

- **Prueba de Escritorio**

radio	angulo	area_sector	cuadrado_area
14.69	39.52	74.4231	5538.7921
85.27	90	5710.6039	32610997.3393

- **Codificación en C**

```c
#include <stdio.h>

int main(void) {
    double PI = 3.14159;
    double FACTOR = 360.0;
    double radio, angulo, area_sector, cuadrado_area = 0.0;

    printf("Ingrese el radio del sector circular: ");
    scanf("%lf", &radio);

    printf("Ingrese el angulo del sector circular en grados: ");
    scanf("%lf", &angulo);

    if (radio > 0 && angulo > 0) {
        area_sector = (PI * radio * radio * angulo) / FACTOR;
        cuadrado_area = area_sector * area_sector;

        printf("Operacion realizada: Calculo de area de sector circular y su cuadrado\n");
        printf("El area del sector es: %.4lf\n", area_sector);
        printf("El cuadrado del resultado principal es: %.4lf\n", cuadrado_area);
    } else {
        printf("\nError: El radio y el angulo deben ser estrictamente mayores a cero.\n");
    }

    printf("Radio ingresado: %.2lf\n", radio);
    printf("Angulo ingresado: %.2lf grados\n", angulo);

    return 0;
}
```

Fig. 16. Código del programa "sector.c"

- **Salida del programa**

```
C:\java_project\primer-ciclo-2026_2027\codigosC>gcc sector.c -o sector

C:\java_project\primer-ciclo-2026_2027\codigosC>sector
Ingrese el radio del sector circular: 5.26
Ingrese el angulo del sector circular en grados: 45
Operacion realizada: Calculo de area de sector circular y su cuadrado
El area del sector es: 10.8650
El cuadrado del resultado principal es: 118.0489
Radio ingresado: 5.26
Angulo ingresado: 45.00 grados
```

Fig. 17. compilación y ejecución.

- **Conclusiones**

1. Gracias a que definimos el valor de pi como una contante (PI=3.14159), pudimos limitar la cantidad de decimales que este usaría para que los resultados sean precisos.
2. La fórmula geométrica implementada permitió resolver de manera directa un problema de trigonometría aplicada sin sobrecargar el procesador con iteraciones.
3. La condición (radio > 0 && angulo > 0) nos sirvió para impedir el procesamiento de medidas geométricas inválidas.

- **Enunciado del Algoritmo**

TRABAJO PRÁCTICO: DESARROLLO DE ALGORITMOS

Diagrama de flujo • Pseudocódigo • Prueba de escritorio • Codificación en C • Conclusiones

Ejercicio 106 – Nivel Medio

En una empresa de telecomunicaciones, construya un algoritmo que lea un peso en onzas (real). Use una constante simbolica ONZA_GRAMO = 28.3495. Calcule la equivalencia en gramos con la expresion aritmetica (onzas * ONZA_GRAMO) y asignela a gramos. Evalúe la expresion relacional (gramos > 0). Muestre por pantalla 'X onzas equivalen a Y gramos'. Adicionalmente, calcule la mitad del resultado obtenido. Incluya una seccion que explique brevemente el proposito del algoritmo.

- ## Análisis del Enunciado

Datos de Entrada:

Constante para la obtener la mitad (MITAD) = 2 - Entero
Constante para la conversión (ONZA_GRAMO) = 28.3495 - Real
Onzas (onzas) – Real

Procesos:

gramos = onzas * ONZA_GRAMO
Comprobar si gramos > 0 para validar que el peso ingresado sea positivo y real.
mitad_gramos = gramos / MITAD

Datos de Salida:

Gramos (gramos) - Real
Mitad de los gramos calculados (mitad_gramos) - Real

- ## Pseudocódigo

```
#Inicio
MITAD = 2;
ONZA_GRAMO = 28.3495;
Leer onzas;
gramos = 0.0;
mitad_gramos = 0.0;
es_valido = 0;

Si onzas > 0
gramos = onzas * ONZA_GRAMO;
mitad_gramos = gramos / MITAD;
es_valido = 1;

Si no, gramos = 0.0;
mitad_gramos = 0.0;
es_valido = 0;

Si es_valido == 1
Presentar: (onzas + " onzas equivalen a " + gramos + " gramos")
Presentar: ("La mitad del resultado obtenido es: " + mitad_gramos)
Si no, Presentar: ("Error: El peso en onzas debe ser mayor a cero.")
Fin
```

- ## Prueba de Escritorio

onzas	gramos	mitad_gramos
24.62	697.9647	348.9823
36.19	1025.9684	512.9842

- ## Codificación en C



Fig. 18. Código del programa "gramos.c"

- ## Salida del programa

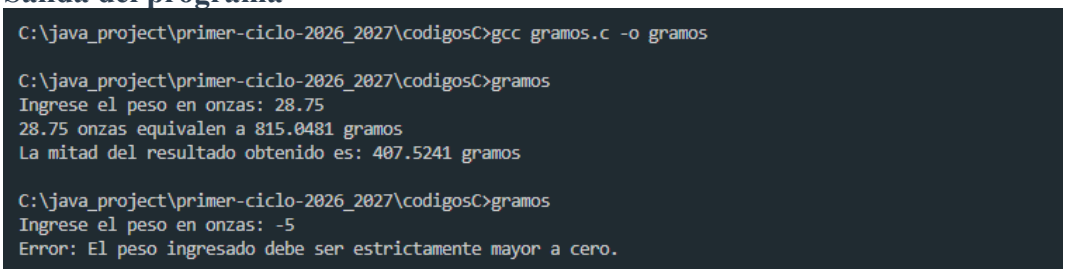


Fig. 19. Compilación y ejecución.

- ## Conclusiones

1. La condición onzas > 0 nos sirvió para filtrar datos erróneos o negativos
2. Si bien no era necesario definir la constante para calcular la mitad de los gramos; sirve para especificar la función de esta constante. Por lo que no importaría si le definimos la constante o si colocamos el número entero.

- ## Enunciado del Algoritmo

Ejercicio 209 – Nivel Medio

En una farmacia, cree un algoritmo que lea la masa (real, en gramos) y el volumen (real, en cm3) de un objeto. Calcule la densidad con la expresion aritmetica (masa / volumen) y asignela a densidad. Evalúe la expresion logica (volumen > 0). Muestre por pantalla la densidad del objeto. Identifique el tipo de dato de cada variable. Ademas del calculo principal, calcule el logaritmo en base 10 del resultado. Agregue al final un resumen de los datos ingresados.

TRABAJO PRÁCTICO: DESARROLLO DE ALGORITMOS

Diagrama de flujo • Pseudocódigo • Prueba de escritorio • Codificación en C • Conclusiones

- ## Análisis del Enunciado

Datos de Entrada:

Masa (masa) - Real
Volumen (volumen) – Real

Procesos:

Evaluar la expresión relacional volumen > 0 para garantizar que el denominador sea válido.
densidad = masa / volumen
log_densidad = log10(densidad)

Datos de Salida:

Densidad (densidad) - Real
Logaritmo de la densidad (log_densidad) – Real

- ## Pseudocódigo

```
#Inicio
Leer masa;
Leer volumen;
densidad = 0.0;
log_densidad = 0.0;
Si volumen > 0
densidad = masa / volumen;
Si densidad > 0
log_densidad = log10(densidad);
Si no log_densidad = 0.0;

Presentar: ("La densidad del objeto es: " + densidad)
Presentar: ("El logaritmo base 10 de la densidad es: " + log_densidad)
Si volumen <= 0
Presentar: ("Error: El volumen debe ser estrictamente mayor a cero.")
Presentar: ("Masa ingresada (gramos): " + masa)
Presentar: ("Volumen ingresado (cm3): " + volumen)
Fin
```

- ## Prueba de Escritorio

masa	volumen	densidad	log_densidad
91	58	1.569	0.1956
26.11	8.2	3.1841	0.503

- ## Codificación en C

TRABAJO PRÁCTICO: DESARROLLO DE ALGORITMOS

Diagrama de flujo • Pseudocódigo • Prueba de escritorio • Codificación en C • Conclusiones

```c
#include <stdio.h>
#include <math.h> // Necesario para la función log10()

int main(void) {

    double masa, volumen, densidad, log_densidad = 0.0;

    printf("Ingrese la masa del objeto en gramos: ");
    scanf("%lf", &masa);
    printf("Ingrese el volumen del objeto en cm3: ");
    scanf("%lf", &volumen);

    if (volumen > 0) {
        densidad = masa / volumen;

        if (densidad > 0) {
            log_densidad = log10(densidad);
        } else {
            log_densidad = 0.0;
        }
        printf("Operacion realizada: Calculo de densidad y logaritmo base 10\n");
        printf("La densidad del objeto es: %.4lf g/cm3\n", densidad);
        printf("El logaritmo base 10 de la densidad es: %.4lf\n", log_densidad);
    } else {
        printf("\nError: El volumen debe ser estrictamente mayor a cero para evitar una division por cero.\n");
    }

    return 0;
}
```

Fig. 20. Código del programa "densidad.c"

- **Salida del programa**

```
C:\java_project\primer-ciclo-2026_2027\codigosC>gcc densidad.c -o densidad

C:\java_project\primer-ciclo-2026_2027\codigosC>densidad
Ingrese la masa del objeto en gramos: 755
Ingrese el volumen del objeto en cm3: 216
Operacion realizada: Calculo de densidad y logaritmo base 10
La densidad del objeto es: 3.4954 g/cm3
El logaritmo base 10 de la densidad es: 0.5435
Masa ingresada: 755.00 gramos
Volumen ingresado: 216.00 cm3
```

Fig. 21. Compilación y ejecución.

- **Conclusiones**

1. La inclusión de la función log10() procedente de <math.h> nos sirvió para obtener de forma directa el logaritmo en base 10 del resultado de la densidad sin requerir algoritmos complejos.
2. Gracias a la evaluación de la expresión lógica volumen > 0, el programa evita colapsar o generar errores críticos de hardware al intentar dividir entre cero.

- **Enunciado del Algoritmo**

Ejercicio 524 – Nivel Medio

En un supermercado, cree un algoritmo que lea un peso en kilogramos (real). Use la constante literal 1000. Calcule la equivalencia en gramos con la expresion aritmetica (kg * 1000) y asignela a gramos. Evalúe la expresion logica (gramos >= 0). Muestre por pantalla 'X kg equivalen a Y gramos'. Adicionalmente, calcule la raiz cuadrada del valor absoluto del resultado. Agregue al inicio una descripcion breve de los pasos a seguir.

- **Análisis del Enunciado**
  Datos de Entrada:

FACT_CONV: 1000 - Entero
Kilogramo (kg) – Real

Procesos:

gramos = kg * FACT_CONV
gramos >= 0
raíz = (valor_absoluto * gramos)-1/2

Datos de Salida:

Gramos (gramos) - Real
raiz – Real

- ## Pseudocódigo

```
#Inicio
FACTOR_CONV = 1000;
Leer kg;
gramos = 0.0;
raiz = 0.0;
Si gramos >= 0
gramos = kg * FACTOR_CONV;
raiz = sqrt(valor_absoluto * gramos);
Si no Presentar: ("Error: El peso en kilogramos no puede ser negativo.")
Presentar: ("Operación realizada: Conversión de kg a gramos y cálculo
de raíz")
Presentar: (kg + " kg equivalen a " + gramos + " gramos")
Presentar: ("La raíz cuadrada del valor absoluto es: " + raiz)
Fin
```

- ## Prueba de Escritorio

kilogramos	gramos	raiz
211.26	211260.00	459.63
742	742000.00	861.39

- ## Codificación en C

TRABAJO PRÁCTICO: DESARROLLO DE ALGORITMOS

Diagrama de flujo • Pseudocódigo • Prueba de escritorio • Codificación en C • Conclusiones

```c
#include <stdio.h>
#include <math.h>

int main() {
    int FACTOR_CONV = 1000;
    double kg, gramos, raiz = 0.0;

    printf("Ingrese el peso en kilogramos: ");
    scanf("%lf", &kg);

    printf("Operacion realizada: Conversion de kg a gramos y calculo de raiz\n");

    if (kg >= 0.0) {
        gramos = kg * FACTOR_CONV;
        raiz = sqrt(fabs(gramos));
        printf("%.2lf kg equivalen a %.2lf gramos\n", kg, gramos);
        printf("La raiz cuadrada del valor absoluto es: %.2lf\n", raiz);
    }

    else {
        printf("Error: El peso en kilogramos debe ser mayor o igual a cero.\n");
        printf("No se pudieron procesar los resultados debido a un valor invalido.\n");
    }

    return 0;
}
```

Fig. 22. Código del programa "densidad.c"

- ## Salida del programa

```
C:\java_project\primer-ciclo-2026_2027\codigosC>gcc kgramos.c -o kilo

C:\java_project\primer-ciclo-2026_2027\codigosC>kilo
Ingrese el peso en kilogramos: 242.13
Operacion realizada: Conversion de kg a gramos y calculo de raiz
242.13 kg equivalen a 242130.00 gramos
La raiz cuadrada del valor absoluto es: 492.07
```

Fig. 23. Compilación y ejecución.

- ## Conclusiones

1. Se requirió la inclusión de la librería matemática para utilizar funciones integradas como sqrt() (raíz cuadrada) y fabs() (valor absoluto), fundamentales para procesar operaciones aritméticas avanzadas en C sin implementar algoritmos manuales complejos.
2. La incorporación de la estructura condicional al inicio asegura que el programa verifique la veracidad de los datos de entrada (kg >= 0), previniendo fallos de ejecución o interpretaciones lógicas erróneas.

- ## Enunciado del Algoritmo

Ejercicio 306 – Nivel Medio

En un hotel, construya un algoritmo que lea el nombre de un estudiante (cadena) y tres notas (reales entre 0 y 10). Use una constante simbólica NOTA_MIN = 7.0. Calcule el promedio con la expresion aritmetica ((nota1 + nota2 + nota3) / 3) y asignelo a promedio. Evalúe la expresión relacional (promedio >= NOTA_MIN). Muestre por pantalla el nombre del estudiante, su promedio y si aprobo o reprobo. Adicionalmente, calcule el inverso del resultado si este no es cero. Incluya una verificacion de rango para cada variable de entrada.

- ## Análisis del Enunciado
  ### Datos de Entrada:

NOTA_MIN (NOTA_MIN) = 7 - Real

DIVISOR (DIVISOR) = 3 - Entero
Variables:
nombre del estudiante (nombre) - Cadena de caracteres
nota 1 (nota1) - Real
nota 2 (nota2) - Real
nota 3 (nota3) - Real

Procesos:

NOTA_MIN (NOTA_MIN) = 7 - Real
DIVISOR (DIVISOR) = 3 - Entero
Variables:
nombre del estudiante (nombre) - Cadena de caracteres
nota 1 (nota1) - Real
nota 2 (nota2) - Real
nota 3 (nota3) – Real

Datos de Salida:

promedio (Real)
inverso del promedio (inverso) - Real
estado de aprobación (estado) - Cadena de caracteres

- ## Pseudocódigo

```
#Inicio
NOTA_MIN = 7.0;
DIVISOR = 3;
Leer nombre;
Leer nota1;
Leer nota2;
Leer nota3;
promedio = 0.0;
inverso = 0.0;
Si nota1 >= 0 y nota1 <= 10 && nota2 >= 0 y nota2 <= 10 && nota3 >= 0 y
nota3 <= 10
promedio = (nota1 + nota2 + nota3) / DIVISOR;
Si promedio >= NOTA_MIN
estado = "Aprobado";
Si no, estado = "Reprobado";
Si promedio != 0
inverso = 1.0 / promedio;
Si no, inverso = 0.0;
Si no, Presentar: ("Error: La nota 3 está fuera del rango permitido (0
a 10).")
Si no, Presentar: ("Error: La nota 2 está fuera del rango permitido (0
a 10).")
Si no, Presentar: ("Error: La nota 1 está fuera del rango permitido (0
a 10).")
Presentar: ("Operación realizada: Cálculo de promedio y estado
académico")
Presentar: ("Estudiante: " + nombre)
Presentar: ("Promedio obtenido: " + promedio)
Presentar: ("Estado: " + estado)
Presentar: ("El inverso del promedio es: " + inverso)
Fin
```

- ## Prueba de Escritorio

nombre	nota1	nota2	nota3	promedio	aprueba	inverso

pablo	3	6	6	5	reprobado	0.20
luisa	4.75	10	6.5	7.08	aprobado	0.1412

- **Codificación en C**

inverso.c U

inverso.c > ...

```c
#include <stdio.h>

int main() {
    double NOTA_MIN = 7.0;
    int DIVISOR = 3;
    char nombre[50];
    double nota1, nota2, nota3, promedio, inverso = 0.0;

    printf("Ingrese el nombre del estudiante: ");
    scanf(" %[^\n]s", nombre);
    printf("Ingrese la nota 1: ");
    scanf("%lf", &nota1);
    printf("Ingrese la nota 2: ");
    scanf("%lf", &nota2);
    printf("Ingrese la nota 3: ");
    scanf("%lf", &nota3);

    printf("\nOperacion realizada: Calculo de promedio y estado academico\n");
    printf("Estudiante: %s\n", nombre);

    if (nota1 >= 0.0 && nota1 <= 10.0) {
        if (nota2 >= 0.0 && nota2 <= 10.0) {
            if (nota3 >= 0.0 && nota3 <= 10.0) {
                promedio = (nota1 + nota2 + nota3) / DIVISOR;

                if (promedio >= NOTA_MIN) {
                    printf("Estado: Aprobado\n");
                } else {
                    printf("Estado: Reprobado\n");
                }

```

Fig. 24. Código del programa "inverso"

```c
                if (promedio != 0.0) {
                    inverso = 1.0 / promedio;
                } else {
                    inverso = 0.0;
                }

                printf("Promedio obtenido: %.2lf\n", promedio);
                printf("El inverso del promedio es: %.4lf\n", inverso);
            } else {
                printf("Error: La nota 3 esta fuera del rango permitido (0 a 10).\n");
            }
        } else {
            printf("Error: La nota 2 esta fuera del rango permitido (0 a 10).\n");
        }
    } else {
        printf("Error: La nota 1 esta fuera del rango permitido (0 a 10).\n");
    }

    return 0;
}

```

Fig. 25. Código del programa "inverso"

- **Salida del programa**

```
C:\java_project\primer-ciclo-2026_2027\codigosC>gcc inverso.c -o inverso

C:\java_project\primer-ciclo-2026_2027\codigosC>inverso
Ingrese el nombre del estudiante: josefina
Ingrese la nota 1: 10
Ingrese la nota 2: 8
Ingrese la nota 3: 7.5

Operacion realizada: Calculo de promedio y estado academico
Estudiante: josefina
Estado: Aprobado
Promedio obtenido: 8.50
El inverso del promedio es: 0.1176
```

Fig. 26. Compilación y ejecución.

- ## Conclusiones

1. El uso de condicionales if-else en cascada permitió filtrar de forma estricta los datos de entrada antes de ejecutar el cálculo matemático, previniendo resultados incoherentes o fallos lógicos en el algoritmo usado.
2. Especificar correctamente los tipos de datos (cadena de caracteres, real, entero) garantiza que el compilador reserve el espacio de memoria adecuado en el hardware y aplique las reglas operativas correctas, como evitar divisiones enteras imprecisas.

- ## Enunciado del Algoritmo

Ejercicio 307 – Nivel Medio

En una empresa de telecomunicaciones, diseñe un algoritmo que lea una cantidad de segundos (entero). Use constantes literales 60 y 3600. Calcule las horas con (segundos / 3600), los minutos con ((segundos % 3600) / 60) y los segundos restantes con (segundos % 60). Asigne cada resultado a variables enteras h, m, s. Muestre por pantalla el formato 'H horas, M minutos y S segundos'. Identifique el tipo de dato de cada variable. Adicionalmente, calcule el seno del angulo resultado (si aplica). Agregue una variable de estado con el codigo de exito (0) al final.

- ## Análisis del Enunciado

Datos de Entrada:

Segundos por hora (SEG_HORA): 3600 - Entero
Segundos por minuto (SEG_MIN): 60 - Entero
Variables:
cantidad de segundos (segundos) – Entero

Procesos:

Se lee el valor de la cantidad de segundos y se valida que sea mayor o igual a cero mediante un condicional.
Si el valor es válido, h = segundos / SEG_HORA
m = segundos % SEG_HORA) / SEG_MIN
s = segundos % SEG_MIN
seno_resultado = Sen(h + m + s)

Datos de Salida:

Cantidad de horas (h) - Entero
Cantidad de minutos (m) - Entero
Segundos restantes (s) - Entero
seno calculado (seno_resultado) - Real

- ## Pseudocódigo

```
#Inicio
```

```
SEG_HORA = 3600;
SEG_MIN = 60;
Leer segundos;
h = 0;
m = 0;
s = 0;
seno_resultado = 0.0;

Si segundos >= 0
    h = segundos / SEG_HORA;
    m = (segundos % SEG_HORA) / SEG_MIN;
    s = segundos % SEG_MIN;
    seno_resultado = sin(h + m + s);
Si no
    Presentar: ("Error: La cantidad de segundos no puede ser negativa.")
    h = 0;
    m = 0;
    s = 0;
    seno_resultado = 0.0;

Presentar: ("Operación realizada: Conversión de tiempo y cálculo
trigonométrico")
Presentar: (h + " horas, " + m + " minutos y " + s + " segundos")
Presentar: ("Seno del resultado: " + seno_resultado)
Fin
```

- **Prueba de Escritorio**

segundos	horas	minutos	segundos	seno
1210311	336	11	51	0.83
392339	108	58	59	-0.93

- **Codificación en C**

TRABAJO PRÁCTICO: DESARROLLO DE ALGORITMOS

Diagrama de flujo • Pseudocódigo • Prueba de escritorio • Codificación en C • Conclusiones

segundos.c U

```c
#include <stdio.h>
#include <math.h>    // Necesario para la función sin()

int main() {
    int SEG_HORA = 3600;
    int SEG_MIN = 60;
    int segundos, h, m, s = 0;
    double seno_resultado = 0.0;

    printf("Ingrese la cantidad de segundos: ");
    scanf("%d", &segundos);

    if (segundos >= 0) {
        h = segundos / SEG_HORA;
        m = (segundos % SEG_HORA) / SEG_MIN;
        s = segundos % SEG_MIN;
        seno_resultado = sin((h + m + s));

        printf("\nOperacion realizada: Conversion de tiempo y calculo del seno\n");
        printf("%d horas, %d minutos y %d segundos\n", h, m, s);
        printf("Seno del resultado: %.3lf\n", seno_resultado);
    }
    else {
        printf("Error: La cantidad de segundos no puede ser negativa.\n");
    }

    return 0;
}
```

Fig. 27. Código del programa "segundos.c"

- **Salida del programa**

```
C:\java_project\primer-ciclo-2026_2027\codigosC>gcc segundos.c -o segundos

C:\java_project\primer-ciclo-2026_2027\codigosC>segundos
Ingrese la cantidad de segundos: 78432

Operacion realizada: Conversion de tiempo y calculo del seno
21 horas, 47 minutos y 12 segundos
Seno del resultado: -0.994
```

Fig. 28. Compilación y ejecución.

- **Conclusiones**

1. La librería <math.h> es necesario para funciones trigonométricas como sin(), cos(), tan().
2. Mantener variables enteras para el tiempo evita errores de redondeo que arruinarían el cálculo exacto de horas, minutos y segundos.

- **Enunciado del Algoritmo**

Ejercicio 126 – Nivel Medio

En una cafeteria, cree un algoritmo que lea tres lados (reales) de un triangulo. Evalúe las expresiones logicas (lado1 + lado2 > lado3) y (lado1 + lado3 > lado2) y (lado2 + lado3 > lado1). Asigne el resultado de combinar las tres con AND a una variable es_triangulo_valido. Muestre por pantalla si los lados forman un triangulo valido. Justifique cada tipo de dato. Adicionalmente, determine si el resultado es mayor que un umbral predefinido. Incluya una seccion que explique brevemente el proposito del algoritmo.

TRABAJO PRÁCTICO: DESARROLLO DE ALGORITMOS

Diagrama de flujo • Pseudocódigo • Prueba de escritorio • Codificación en C • Conclusiones

- ## Análisis del Enunciado

Datos de Entrada:

Umbral de comparación (UMBRAL): 10.0 (Real)
Lado 1 (lado1) - Real
Lado 2 (lado2) - Real
Lado 3 (lado3) – Real
validación (es_valido) - Entero

Procesos:

perimetro = lado1 + lado2 + lado3
Si perimetro > UMBRAL, entonces supera_umbral = 1, de lo contrario supera_umbral = 0.
es_válido = (lado1 + lado2 > lado3) y (lado1 + lado3 > lado2) y (lado2 + lado3 > lado1)

Datos de Salida:
Perímetro o suma total (perimetro)
Indicador de umbral superado (supera_umbral)

- ## Pseudocódigo

```
#Inicio
UMBRAL = 10.0;
Leer lado1;
Leer lado2;
Leer lado3;
es_valido = 0;
perimetro = 0.0;
supera_umbral = 0;
Presentar: ("Propósito: Validar la existencia geométrica de un triángulo a partir de sus tres lados y evaluar su magnitud frente a un umbral predefinido en la cafetería.")
Si lado1 > 0 y lado2 > 0 y lado3 > 0 Si (lado1 + lado2 > lado3) y (lado1 + lado3 > lado2) y (lado2 + lado3 > lado1)
es_valido = 1;
Si no es_valido = 0;
perimetro = lado1 + lado2 + lado3;
Si perimetro > UMBRAL
supera_umbral = 1;
Si no supera_umbral = 0;
Si es_triangulo_valido == 1
Presentar: ("Los lados ingresados SÍ forman un triángulo válido.")
Si no Presentar: ("Los lados ingresados NO forman un triángulo válido.")
Si supera_umbral == 1
Presentar: ("El perímetro" + perimetro + "supera el umbral predefinido" + UMBRAL)
Si no Presentar: ("El perímetro" + perimetro + "no supera el umbral predefinido" + UMBRAL)
Si no Presentar: ("Error: Todos los lados deben ser mayores que cero.")
Fin
```

- ## Prueba de Escritorio

lado1	lado2	lado3	perímetro	¿Es un triángulo válido?	¿Es mayor que UMBRAL = 10?
6.5	4	3.5	14	sí	sí supera

2	3	4	9	sí	no supera

- **Codificación en C**

umbral.c U

umbral.c > main(void)

```c
#include <stdio.h>

int main(void) {
    int UMBRAL = 10;
    double lado1, lado2, lado3, perimetro = 0.0;
    int es_valido, supera_umbral = 0;

    printf("Ingrese los 3 lados del triangulo (separados por espacios): ");
    scanf("%lf %lf %lf", &lado1, &lado2, &lado3);

    if (lado1 > 0 && lado2 > 0 && lado3 > 0) {
        if ((lado1 + lado2 > lado3) && (lado1 + lado3 > lado2) && (lado2 + lado3 > lado1)) {
            es_valido = 1;
        } else {  //Si alguno de los lados es 0
            es_valido = 0;
        }

        perimetro = lado1 + lado2 + lado3;


        if (perimetro > UMBRAL) {    //Comparación del perímetro con el umbral
            supera_umbral = 1;
        } else {
            supera_umbral = 0;
        }

        if (es_valido == 1) {
            printf("Los lados ingresados forman un triangulo valido.\n");
        } else {
            printf("Los lados ingresados no forman un triangulo valido.\n");
        }

        if (supera_umbral == 1) {
            printf("El perimetro (%.2lf) supera el umbral de %d.\n", perimetro, UMBRAL);
        } else {
            printf("El perimetro (%.2lf) no supera el umbral de %d.\n", perimetro, UMBRAL);
        }

    } else {
        printf("\nError: Todos los lados del triangulo deben ser mayores a cero.\n");
    }

    return 0;
}

```

Fig. 29. Código del programa "umbral.c"

- **Salida del programa**

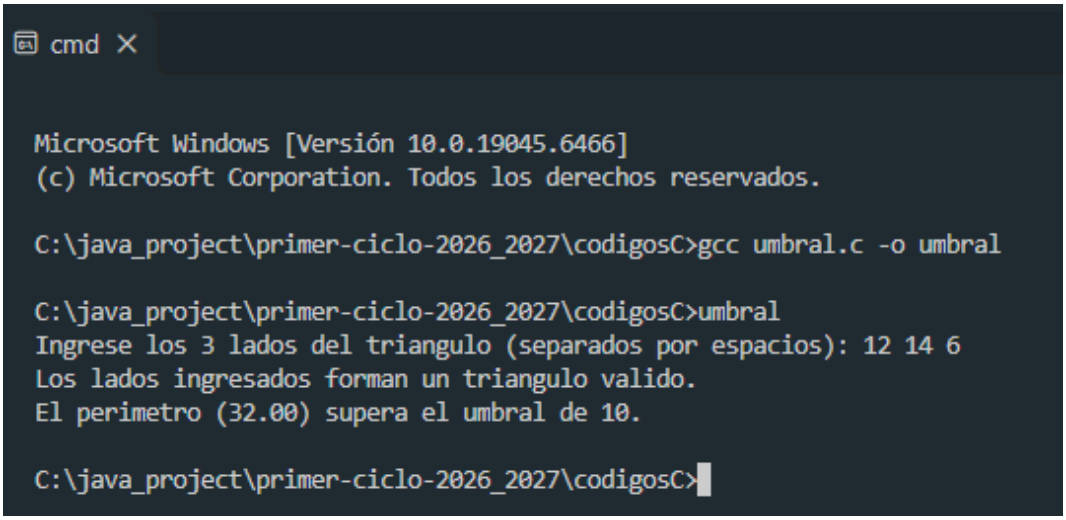


Fig. 30. Compilación y ejecución.

- ## Conclusiones

1. Es importante que, en el código, definamos a los 3 lados y al perímetro como valores reales (double), ya que suelen contener decimales o requerir una alta precisión durante el cálculo.
2. Gracias a la evaluación combinada con el operador lógico &&, el código verifica rigurosamente las tres reglas geométricas necesarias para garantizar la existencia física de un triángulo.
3. Las validaciones iniciales evitan que el programa procese lados que sean iguales o menores a cero, previniendo errores lógicos, asegurándose que el usuario ingrese 3 lados válidos para un triángulo.

- ## Enunciado del Algoritmo

Ejercicio 266 – Nivel Avanzado

En un hospital, diseñe un algoritmo que calcule la tarifa de Parqueadero. Lea las horas (real), el tipo de vehiculo (caracter: 'A' auto, 'M' moto, 'C' camioneta), el dia de la semana (caracter), la modalidad (caracter: 'H' por hora, 'D' dia completo, 'M' mensual) y el descuento VIP (booleano). Use constantes simbolicas PRECIO_AUTO = 2.5, PRECIO_MOTO = 1.5, PRECIO_CAMIONETA = 3.5, DESC_VIP = 0.20, DESC_MENSUAL = 0.30. Use expresiones logicas para seleccionar tarifa. Calcule el costo. Muestre por pantalla el detalle. Ademas del calculo principal, calcule el resto de una division adicional. Agregue al final un resumen de los datos ingresados.

- ## Análisis del Enunciado

Datos de Entrada:

Constante para calcular el precio de tarifa del auto (PRECIO_AUTO) = 2.5 - Real
Constante para calcular el precio de tarifa de la moto (PRECIO_MOTO) = 1.5 - Real
Constante para calcular el precio de tarifa de la camioneta (PRECIO_CAMIONETA) = 3.5 - Real
Descuento por ser usuario VIP (DESC_VIP) = 0.20 - Real
Descuento mensual (DESC_MENSUAL) = 0.30 - Real
Horas de estacionamiento (horas) - Real
Tipo de vehiculo (tipo_vehiculo) - Caracter
Día de la semana (dia) - Caracter
Modalidad de cobro (modalidad) - Caracter
Descuento VIP (es_vip) - Entero

Procesos:

Se valida mediante condicionales anidados que las horas sean mayores a cero y que el tipo de vehículo y modalidad sean válidos.
Según el tipo de vehículo, se selecciona la tarifa base.

Se aplican descuentos por modalidad (M) y por cliente VIP (es_vip == 1) si corresponde.
Se calcula el costo total y se obtiene el residuo de una división adicional usando valores enteros de las horas.

Datos de Salida:

costo total (costo) - Real
residuo adicional (residuo) – Entero

- ## Pseudocódigo

```
#Inicio
PRECIO_AUTO = 2.5;
PRECIO_MOTO = 1.5;
PRECIO_CAMIONETA = 3.5;
DESC_VIP = 0.20;
DESC_MENSUAL = 0.30;
Leer horas;
Leer tipo_vehiculo;
Leer dia;
Leer modalidad;
Leer es_vip;
costo = 0.0;
residuo = 0;
Si horas > 0
Si tipo_vehiculo == 'A'
costo = horas * PRECIO_AUTO;
Si no Si tipo_vehiculo == 'M'
costo = horas * PRECIO_MOTO;
Si no Si tipo_vehiculo == 'C'
costo = horas * PRECIO_CAMIONETA;
Si no costo = 0.0;
Si modalidad == 'M'
costo = costo - (costo * DESC_MENSUAL);
Si es_vip == 1
costo = costo - (costo * DESC_VIP);
residuo = (entero)(horas) % 3;
Si horas <=0
Presentar: ("Error: Las horas deben ser mayores a cero.")

Presentar: ("Operación realizada: Cálculo de tarifa de parqueadero")
Presentar: ("Horas: " + horas)
Presentar: ("Tipo de vehículo: " + tipo_vehiculo)
Presentar: ("Día de la semana: " + dia)
Presentar: ("Modalidad: " + modalidad)
Presentar: ("Es VIP (1=Sí, 0=No): " + es_vip) Fin
Presentar: ("Costo total de parqueadero: " + costo)
Presentar: ("Residuo de división adicional: " + residuo)
Fin
```

- ## Prueba de Escritorio

horas	tipo_vehiculo	dia	modalidad	VIP	costo	residuo
8	M	V	D	No	12	2
160	C	m	M	Sí	313.60	1

- ## Codificación en C



```c
#include <stdio.h>

int main() {
    double PRECIO_AUTO = 2.5;
    double PRECIO_MOTO = 1.5;
    double PRECIO_CAMIONETA = 3.5;
    double DESC_VIP = 0.20;
    double DESC_MENSUAL = 0.30;

    int horas = 0;
    char tipo_vehiculo = ' ';
    char dia = ' ';
    char modalidad = ' ';
    int es_vip = 0;
    double costo = 0.0;
    int residuo = 0;

    printf("Ingrese las horas de estacionamiento: ");
    scanf("%d", &horas);
    printf("Ingrese el tipo de vehiculo ('A': Auto, 'M': Moto, 'C': Camioneta): ");
    scanf(" %c", &tipo_vehiculo);
    printf("Ingrese el dia de la semana ('L', 'M', 'm', 'J', 'V', 'S', 'D'): ");
    scanf(" %c", &dia);
    printf("Ingrese la modalidad ('H': Por hora, 'D': Dia completo, 'M': Mensual): ");
    scanf(" %c", &modalidad);
    printf("Tiene descuento VIP? (1 para Si, 0 para No): ");
    scanf("%d", &es_vip);

    if (horas > 0) {
        if (tipo_vehiculo == 'A') {
            costo = horas * PRECIO_AUTO;
        } else {
            if (tipo_vehiculo == 'M') {
                costo = horas * PRECIO_MOTO;
            } else {
                if (tipo_vehiculo == 'C') {
                    costo = horas * PRECIO_CAMIONETA;
                } else {
                    printf("Error: Tipo de vehiculo no valido.\n");
                }
            }
        }

        if (modalidad == 'M') {
            costo = costo - (costo * DESC_MENSUAL);
            } else {
                if (modalidad == 'D') {
                    costo = costo;
                } else {
                    if (modalidad == 'H') {
                        costo = costo;
                    } else {
                        printf("Error: Modalidad no valida.\n");
                    }
                }
        }

        if (es_vip == 1) {
            costo = costo - (costo * DESC_VIP);
        }

        if (es_vip == 1) {
            costo = costo - (costo * DESC_VIP);
        }

        residuo = ((int)(horas) % 3);

        printf("Horas: %d\n", horas);
        printf("Tipo de vehiculo: %c\n", tipo_vehiculo);
        printf("Dia de la semana: %c\n", dia);
        printf("Modalidad: %c\n", modalidad);
        printf("Es VIP (1=Si, 0=No): %d\n", es_vip);

        printf("\nOperacion realizada: Calculo de tarifa de parqueadero\n");
        printf("Costo total de parqueadero: $%.2lf\n", costo);
        printf("Residuo de division adicional: %d\n", residuo);

    }
    else {
        printf("Error: Las horas deben ser mayores a cero.\n");
    }

    return 0;
}
```

Fig. 31. Código "parqueadero.c"

- ## Salida del programa

```
C:\java_project\primer-ciclo-2026_2027\codigosC>gcc parqueadero.c -o VIP

C:\java_project\primer-ciclo-2026_2027\codigosC>VIP
Ingrese las horas de estacionamiento: 8
Ingrese el tipo de vehiculo ('A': Auto, 'M': Moto, 'C': Camioneta): A
Ingrese el dia de la semana ('L', 'M', 'm', 'J', 'V', 'S', 'D'): m
Ingrese la modalidad ('H': Por hora, 'D': Dia completo, 'M': Mensual): M
Tiene descuento VIP? (1 para Si, 0 para No): 0
Horas: 8
Tipo de vehiculo: A
Dia de la semana: m
Modalidad: M
Es VIP (1=Si, 0=No): 0

Operacion realizada: Calculo de tarifa de parqueadero
Costo total de parqueadero: $14.00
Residuo de division adicional: 2
```

Fig. 32. Compilación y ejecución.

- ## Conclusiones

- Usar una validación estricta de caracteres en mayúsculas ('A', 'M', 'C') limpia bastante las expresiones lógicas del código.
- El casteo explícito a entero para sacar el módulo evita errores de compilación con datos reales.

- ## Enunciado del Algoritmo

Ejercicio 267 – Nivel Avanzado

En una tienda de ropa, construya un algoritmo que clasifique un credito. Lea el monto (real), el plazo (entero, meses), la tasa (real), el tipo (caracter: 'C' consumo, 'H' hipotecario, 'V' vehicular, 'E' estudio) y la garantia (real). Use constantes simbolicas MONTO_MIN = 1000, PLAZO_MAX = 60, TASA_MAX = 0.36, COBERTURA_GARANTIA = 1.5. Evalúe las expresiones logicas para clasificar el riesgo. Asigne la categoria de riesgo a una cadena. Muestre por pantalla la clasificacion. Ademas del calculo principal, calcule el resto de una division adicional. Asegurese de que las expresiones logicas esten bien parentizadas

- ## Análisis del Enunciado

Datos de Entrada:

MONTO_MIN (MONTO_MIN): 1000.0 - Real
PLAZO_MAX (PLAZO_MAX): 60 - Entero
TASA_MAX (TASA_MAX): 0.36 - Real
COBERTURA_GARANTIA (COBERTURA_GARANTIA): 1.5 - Real
monto del crédito (monto) - Real
plazo en meses (plazo) - Entero
tasa de interés (tasa) - Real
tipo de crédito (tipo) - Caracter
garantía aportada (garantia) – Real

Procesos:

Se leen los datos correspondientes a monto, plazo, tasa, tipo de crédito y garantía.
Se valida mediante expresiones lógicas que el monto sea mayor o igual a MONTO_MIN, el plazo no supere a PLAZO_MAX, y la tasa esté dentro del límite permitido (TASA_MAX).
Dependiendo del tipo de crédito y las condiciones de la garantía, se evalúa el riesgo
Luego, se asigna la categoría correspondiente ("Bajo", "Medio" o "Alto") mediante condicionales anidados.
Se calcula el residuo de una división adicional utilizando datos enteros, como el plazo y un divisor (12).

Datos de Salida:

categoría de riesgo (categoria) - Cadena de caracteres
residuo (residuo) - Entero

- ## Pseudocódigo

```
#Inicio
MONTO_MIN = 1000.0;
PLAZO_MAX = 60;
TASA_MAX = 0.36;
COBERTURA_GARANTIA = 1.5;
Leer monto;
Leer plazo;
Leer tasa;
Leer tipo;
Leer garantia;
residuo = 0;
Si ((monto >= MONTO_MIN) y (plazo > 0))
Si ((plazo <= PLAZO_MAX) y (tasa <= TASA_MAX))
Si ((tipo == 'C') o (tipo == 'H') o (tipo == 'V') o (tipo == 'E'))
Si (garantia >= (monto / COBERTURA_GARANTIA))
categoria = "Riesgo Bajo";
Si no, categoria = "Riesgo Medio";
Si no, categoria = "Tipo inválido";
Si no, categoria = "Rechazado por plazo o tasa";
Si no categoria = "Rechazado por monto inválido";
residuo = plazo % 12;
Presentar: ("Operación realizada: Clasificación de crédito")
Presentar: ("Clasificación de riesgo: " + categoria)
Presentar: ("Residuo de división adicional (plazo % 12): " + residuo)
Fin
```

- ## Prueba de Escritorio

monto	plazo	tasa	tipo	garantía	característica	residuo
10000	36	0.20	H	4000	Riesgo medio	0
3000	24	0.12	V	3000	Riesgo bajo	0

TRABAJO PRÁCTICO: DESARROLLO DE ALGORITMOS

Diagrama de flujo • Pseudocódigo • Prueba de escritorio • Codificación en C • Conclusiones

Codificación en C

```c
#include <stdio.h>

int main() {
    double MONTO_MIN = 1000.0;
    int PLAZO_MAX = 60;
    double TASA_MAX = 0.36;
    double COBERTURA_GARANTIA = 1.5;
    double monto, tasa, garantia = 0.0;
    int plazo, residuo = 0;
    char tipo = ' ';

    printf("Ingrese el monto del credito: ");
    scanf("%lf", &monto);
    printf("Ingrese el plazo en meses: ");
    scanf("%d", &plazo);
    printf("Ingrese la tasa de interes: ");
    scanf("%lf", &tasa);
    printf("Ingrese el tipo de credito ('C': Consumo, 'H': Hipotecario, 'V': Vehicular, 'E': Estudio): ");
    scanf(" %c", &tipo);
    printf("Ingrese el valor de la garantia: ");
    scanf("%lf", &garantia);

    if ((monto >= MONTO_MIN) && (plazo > 0)) {
        if ((plazo <= PLAZO_MAX) && (tasa <= TASA_MAX)) {
            if ((tipo == 'C') || (tipo == 'H') || (tipo == 'V') || (tipo == 'E')) {
                if (garantia >= (monto / COBERTURA_GARANTIA)) {
                    printf("Riesgo Bajo \n");
                } else {
                    printf("Riesgo Medio \n");
                }
            } else {
                printf("Tipo de credito invalido \n");
            }
        } else {
            printf("Rechazado por plazo o tasa \n");
        }
    } else {
        printf("Rechazado por monto invalido \n");
    }

    residuo = plazo % 12;

    printf("\nOperacion realizada: Clasificacion de credito\n");
    printf("Residuo de division adicional (plazo %% 12): %d\n", residuo);

    return 0;
}
```

Fig. 33. Código "crédito.c"

- ## Salida del programa

```
C:\java_project\primer-ciclo-2026_2027\codigosC>credito
Ingrese el monto del credito: 1500
Ingrese el plazo en meses: 12
Ingrese la tasa de interes: 0.15
Ingrese el tipo de credito ('C': Consumo, 'H': Hipotecario, 'V': Vehicular, 'E': Estudio): C
Ingrese el valor de la garantia: 1200
Riesgo Bajo

Operacion realizada: Clasificacion de credito
Residuo de division adicional (plazo % 12): 0
```

Fig. 34. Compilación y ejecución.

- ## Conclusiones

1. El operador módulo aplicado sobre variables enteras como el plazo facilita obtener indicadores de tiempo útiles, como los meses restantes fuera de un año completo.

2. El uso correcto de paréntesis en expresiones lógicas complejas evita ambigüedades en la precedencia de operadores y asegura que C evalúe las condiciones en el orden deseado.

- ## Enunciado del Algoritmo

Ejercicio 268 – Nivel Avanzado

En una farmacia, cree un algoritmo que calcule los costos de un servicio de salud. Lea la edad (entero), el tipo de plan (caracter: 'B' basico, 'C' completo, 'P' premium), el numero de dependientes (entero), el historial medico (caracter: 'S' sin condiciones, 'C' condiciones cronicas, 'P' preexistente) y la frecuencia de uso (entero, veces/mes). Use constantes simbolicas PLAN_B = 100, PLAN_C = 250, PLAN_P = 500, RECARGO_CRONICO = 1.5, RECARGO_PREEXISTENTE = 2.0. Calcule el costo mensual. Muestre por pantalla. Ademas del calculo principal, calcule el resto de una division adicional. Identifique explicitamente los datos de entrada y la informacion de salida.

- ## Análisis del Enunciado

Datos de Entrada:

Plan B (PLAN_B): 100 - Entero
Plan C (PLAN_C): 250 - Entero
Plan P (PLAN_P): 500 - Entero
Recargo crónico (RECARGO_CRONICO): 1.5 - Real
Recargo preexistente (RECARGO_PREEXISTENTE): 2.0 - Real
Edad del usuario (edad) - Entero
Tipo de plan (tipo_plan) - Caracter
Número de dependientes (dependientes) - Entero
Historial médico (historial) - Caracter
Frecuencia de uso (frecuencia) - Entero

Procesos:

Se validan los datos de entrada para asegurar que la edad, dependientes y frecuencia sean mayores o iguales a cero.
Se selecciona el costo base dependiendo del tipo de plan escogido ('B', 'C', 'P').
Aplicación de recargos al costo base si el historial médico muestra condiciones crónicas ('C') o preexistentes ('P').
Se suma un valor adicional por cada dependiente y por la frecuencia de uso mensual.
Cálculo del residuo de una división adicional utilizando un dato entero (como la edad módulo 5).

Datos de Salida:

costo mensual del servicio (costo_mensual) - Real
residuo de division adicional (residuo) – Entero

- ## Pseudocódigo

```
#Inicio
PLAN_B = 100.0;
PLAN_C = 250.0;
PLAN_P = 500.0;
RECARGO_CRONICO = 1.5;
RECARGO_PREEXISTENTE = 2.0;
Leer edad;
Leer tipo_plan;
Leer dependientes;
Leer historial;
Leer frecuencia;
costo_mensual = 0.0;
residuo = 0;
Si ((edad >= 0) y (dependientes >= 0) y (frecuencia >= 0))
Si (tipo_plan == 'B')
```

```
costo_mensual = PLAN_B;
Si no, Si (tipo_plan == 'C')
costo_mensual = PLAN_C;
Si no, Si (tipo_plan == 'P') c
osto_mensual = PLAN_P;
Si no, costo_mensual = 0.0;
Si (historial == 'C')
costo_mensual = costo_mensual * RECARGO_CRONICO;
Si no, Si (historial == 'P')

costo_mensual = costo_mensual * RECARGO_PREEXISTENTE;
costo_mensual = costo_mensual + (dependientes * 20.0) + (frecuencia *
5.0);
residuo = edad % 5;
Si no, Presentar: ("Error: La edad, dependientes o frecuencia no pueden
ser negativos.")
costo_mensual = 0.0;
residuo = 0;
Presentar: ("Operación realizada: Cálculo de costos de salud")
Presentar: ("Costo mensual total: " + costo_mensual)
Presentar: ("Residuo de división adicional (edad % 5): " + residuo)
Fin
```

- **Prueba de Escritorio**

edad	tipo_plan	dependientes	historial	frecuencia	costo	residuo
82	P	6	P	10	920	2
18	B	1	S	2	130	3

- **Codificación en C**



```c
#include <stdio.h>

int main() {
    int PLAN_B = 100;
    int PLAN_C = 250;
    int PLAN_P = 500;
    double RECARGO_CRONICO = 1.5;
    double RECARGO_PREEXISTENTE = 2;
    int edad, dependientes, frecuencia, residuo = 0;
    char tipo_plan = ' ';
    char historial = ' ';
    double costo_mensual = 0.0;

    printf("Ingrese la edad: ");
    scanf("%d", &edad);
    printf("Ingrese el tipo de plan ('B': Basico, 'C': Completo, 'P': Premium): ");
    scanf(" %c", &tipo_plan);
    printf("Ingrese el numero de dependientes: ");
    scanf("%d", &dependientes);
    printf("Ingrese el historial medico ('S': Sin condiciones, 'C': Cronicas, 'P': Preexistente): ");
    scanf(" %c", &historial);
    printf("Ingrese la frecuencia de uso: ");
    scanf("%d", &frecuencia);

    if ((edad >= 0) && (dependientes >= 0) && (frecuencia >= 0)) {
        if (tipo_plan == 'B') {
            costo_mensual = PLAN_B;
        } else {
            if (tipo_plan == 'C') {
                costo_mensual = PLAN_C;
            } else {
                if (tipo_plan == 'P') {
                    costo_mensual = PLAN_P;
                } else {
                    costo_mensual = 0.0;
                }
            }
        }

        if (historial == 'C') {
            costo_mensual = costo_mensual * RECARGO_CRONICO;
        } else {
            if (historial == 'P') {
                costo_mensual = costo_mensual * RECARGO_PREEXISTENTE;
            } else {
                if (historial == 'S') {
                    costo_mensual = costo_mensual;
                } else {
                    costo_mensual = 0.0;
                }
            }
        }

        costo_mensual = costo_mensual + (dependientes * 20) + (frecuencia * 5);
        residuo = edad % 5;

        printf("\nOperacion realizada: Calculo de costos de salud\n");
        printf("Costo mensual total: $%.2lf\n", costo_mensual);
        printf("Residuo (edad %% 5): %d\n", residuo);
    }
    else {
        printf("Error: La edad, los dependientes y la frecuencia deben ser mayores o iguales a cero.\n");
    }

    return 0;
}
```

Fig. 35. Código del programa "farmacia.c"

- **Salida del programa**

```
C:\java_project\primer-ciclo-2026_2027\codigosC>gcc farmacia.c -o farmacia

C:\java_project\primer-ciclo-2026_2027\codigosC>farmacia
Ingrese la edad: 22
Ingrese el tipo de plan ('B': Basico, 'C': Completo, 'P': Premium): B
Ingrese el numero de dependientes: 2
Ingrese el historial medico ('S': Sin condiciones, 'C': Cronicas, 'P': Preexistente): S
Ingrese la frecuencia de uso: 3

Operacion realizada: Calculo de costos de salud
Costo mensual total: $155.00
Residuo (edad % 5): 2
```

Fig. 36. Compilación y ejecución.

- ## Conclusiones

1. Usar constantes simbólicas facilita mucho al momento de querer cambiar los precios base o los recargos médicos sin tener que buscar número por número en todo el código.
2. Combinar datos enteros con reales en las fórmulas de costos permite mantener la precisión exacta en centavos y recargos porcentuales.

- ## Enunciado del Algoritmo

Ejercicio 269 – Nivel Avanzado

En un taller mecanico, diseñe un algoritmo que recomiende un libro. Lea el genero preferido (cadena), la edad (entero), el nivel de lectura (caracter: 'B' bajo, 'M' medio, 'A' alto), el idioma (cadena) y el tiempo disponible (entero, horas/semana). Use constantes simbolicas EDAD_MIN = 8, EDAD_JUEVEN = 18, EDAD_ADULTO = 30, HORAS_MIN = 2, HORAS_MAX = 10. Evalúe las expresiones logicas para personalizar la recomendacion. Asigne el libro recomendado a una cadena. Muestre por pantalla la recomendacion. Ademas del calculo principal, calcule el resto de una division adicional. Asegurese de inicializar todas las variables antes de usarlas.

- ## Análisis del Enunciado

Datos de Entrada:

EDAD_MIN (EDAD_MIN): 8 - Entero
EDAD_JUEVEN (EDAD_JUEVEN): 18 - Entero
EDAD_ADULTO (EDAD_ADULTO): 30 - Entero
HORAS_MIN (HORAS_MIN): 2 - Entero
HORAS_MAX (HORAS_MAX): 10 - Entero
genero preferido (genero) - Cadena de caracteres
edad del usuario (edad) - Entero
nivel de lectura (nivel) - Caracter
idioma de preferencia (idioma) - Cadena de caracteres
tiempo disponible (tiempo) - Entero

Procesos:

Se validan los datos de género, edad, nivel de lectura, idioma y tiempo disponible mediante expresiones lógicas si la edad y el tiempo cumplen con los mínimos establecidos (EDAD_MIN y HORAS_MIN).
Mediante condicionales anidados, se evalúa la edad y el perfil del usuario para asignar una recomendación de libro específica a la variable de cadena (libro).
Se calcula el residuo de una división adicional utilizando un valor entero (edad % 4).

Datos de Salida:

libro recomendado (libro) - Cadena de caracteres
residuo de division (residuo) - Entero

TRABAJO PRÁCTICO: DESARROLLO DE ALGORITMOS

Diagrama de flujo • Pseudocódigo • Prueba de escritorio • Codificación en C • Conclusiones

- ## Pseudocódigo

```
#Inicio
EDAD_MIN = 8;
EDAD_JOVEN = 18;
EDAD_ADULTO = 30;
HORAS_MIN = 2;
HORAS_MAX = 10;
genero = "";
edad = 0;
nivel = ' ';
idioma = "";
tiempo = 0;
residuo = 0;
Leer genero;
Leer edad;
Leer nivel;
Leer idioma;
Leer tiempo;
Si ((edad >= EDAD_MIN) y (tiempo >= HORAS_MIN))
Si (edad < EDAD_JOVEN)
libro = "Cuentos juveniles de aventura";
Si no, Si (edad < EDAD_ADULTO)
libro = "Novela contemporanea y ciencia ficcion";
Si no libro = "Ensayo y literatura clasica";
residuo = edad % 4;
Si no, libro = "Libro ilustrado infantil o cuentos cortos";
residuo = 0;
Presentar: ("Operacion realizada: Recomendacion de libro personalizada")
Presentar: ("Libro recomendado: " + libro)
Presentar: ("Residuo de division adicional (edad % 4): " + residuo)
Fin
```

- ## Prueba de Escritorio

genero	edad	nivel	idioma	tiempo	recomendación	residuo
Comedia	6	B	Español	5	Libro ilustrado infantil o cuentos cortos	0
Drama	32	A	Inglés	4	Ensayo y literatura clásica	0

- ## Codificación en C



```c
#include <stdio.h>

int main() {
    int EDAD_MIN = 8;
    int EDAD_JUEVEN = 18;
    int EDAD_ADULTO = 30;
    int HORAS_MIN = 2;
    int HORAS_MAX = 10;

    char genero[30] = "";
    int edad, tiempo, residuo = 0;
    char nivel = ' ';
    char idioma[20] = "";

    printf("Ingrese el genero preferido: ");
    scanf(" %[^\n]s", genero);
    printf("Ingrese la edad: ");
    scanf("%d", &edad);
    printf("Ingrese el nivel de lectura ('B': bajo, 'M': medio, 'A': alto): ");
    scanf(" %c", &nivel);
    printf("Ingrese el idioma: ");
    scanf(" %[^\n]s", idioma);
    printf("Ingrese el tiempo disponible (horas/semana): ");
    scanf("%d", &tiempo);

    printf("\nOperacion realizada: Recomendacion de libro personalizada\n");

    if ((edad >= EDAD_MIN) && (tiempo >= HORAS_MIN) && (tiempo <= HORAS_MAX)) {
        if (edad < EDAD_JUEVEN) {
            printf("Cuentos juveniles de aventura");
        } else {
            if (edad < EDAD_ADULTO) {
                printf("Novela contemporanea y ciencia ficcion \n");
            } else {
                printf("Ensayo y literatura clasica \n");
            }
        }
        residuo = edad % 4;
    }
    else {
        printf("Libro ilustrado infantil o cuentos cortos \n");
        residuo = 0;
    }

    printf("Residuo de division adicional (edad %% 4): %d\n", residuo);

    return 0;
}
```

Fig. 37. Código "libro.c".

- **Salida del programa**

```
C:\java_project\primer-ciclo-2026_2027\codigosC>gcc libro.c -o libro

C:\java_project\primer-ciclo-2026_2027\codigosC>libro
Ingrese el genero preferido: Fantasía
Ingrese la edad: 18
Ingrese el nivel de lectura ('B': bajo, 'M': medio, 'A': alto): B
Ingrese el idioma: Español
Ingrese el tiempo disponible (horas/semana): 2

Operacion realizada: Recomendacion de libro personalizada
Novela contemporanea y ciencia ficcion
Residuo de division adicional (edad % 4): 2
```

Fig. 38. Compilación y ejecución.

- ## Conclusiones

1. Inicializar todas las variables al principio previene valores basura en memoria y garantiza un comportamiento estable del programa.
2. Es importante que no coloquemos tildes al usar el lenguaje de C, ya que, al compilar y ejecutar el programa, no muestra las tildes correctamente.

- ## Enunciado del Algoritmo

Ejercicio 270 – Nivel Avanzado

En una escuela, construya un algoritmo que calcule la tarifa de transporte privado. Lea la distancia (real, km), el tiempo (real, minutos), el tipo de servicio (caracter: 'T' taxi, 'U' uber, 'L' lujo), la demanda (caracter: 'B' baja, 'N' normal, 'A' alta) y la hora (entero 0-23). Use constantes simbolicas TARIFA_BASE = 2.0, COSTO_KM = 1.0, COSTO_MIN = 0.25, FACTOR_UBER = 1.2, FACTOR_LUJO = 2.0, FACTOR_DEMANDA_ALTA = 1.5. Calcule la tarifa. Muestre por pantalla. Ademas del calculo principal, calcule el resto de una division adicional. Incluya una variable auxiliar para llevar un contador de operaciones.

- ## Análisis del Enunciado

Datos de Entrada:

Tarifa base (TARIFA_BASE): 2.0 - Real
Costo KM (COSTO_KM): 1.0 - Real
Costo mínimo (COSTO_MIN): 0.25 - Real
Factor de uber (FACTOR_UBER): 1.2 - Real
Factor de lujo (FACTOR_LUJO): 2.0 - Real
Factor por demanda alta (FACTOR_DEMANDA_ALTA): 1.5 - Real
Variables:
distancia del recorrido (distancia) - Real
tiempo del recorrido (tiempo) - Real
tipo de servicio (servicio) - Caracter
nivel de demanda (demanda) - Caracter
hora del día (hora) - Entero

Procesos:

Se valida los datos de entrada: distancia, tiempo, servicio, demanda y hora mediante expresiones lógicas que la distancia y el tiempo sean mayores a cero, y que la hora esté en el rango de 0 a 23.
Se incrementa el contador de operaciones por cada validación o cálculo realizado (contador = contador + 1).

TRABAJO PRÁCTICO: DESARROLLO DE ALGORITMOS

Diagrama de flujo • Pseudocódigo • Prueba de escritorio • Codificación en C • Conclusiones

Cálculo de la tarifa base sumando la tarifa fija más el producto de la distancia por su costo y el tiempo por su costo.
Mediante condicionales anidados, se aplica un factor multiplicador según el tipo de servicio ('U' o 'L') y si la demanda es alta, se pone 'A'.
Cálculo del residuo de una división adicional usando valores enteros (hora % 6).

Datos de Salida:

tarifa total (tarifa) - Real
residuo de division adicional (residuo) - Entero
contador de operaciones (contador) - Entero

- ## Pseudocódigo

```
#Inicio
TARIFA_BASE = 2.0;
COSTO_KM = 1.0;
COSTO_MIN = 0.25;
FACTOR_UBER = 1.2;
FACTOR_LUJO = 2.0;
FACTOR_DEMANDA_ALTA = 1.5;
distancia = 0.0;
tiempo = 0.0;
servicio = ' ';
demanda = ' ';
hora = 0;
tarifa = 0.0;
residuo = 0;
contador = 0;
Leer distancia;
Leer tiempo;
Leer servicio;
Leer demanda;
Leer hora;
Si ((distancia > 0) y (tiempo > 0) y (hora >= 0) y (hora <= 23))
contador = contador + 1;
tarifa = TARIFA_BASE + (distancia * COSTO_KM) + (tiempo * COSTO_MIN);
Si (servicio == 'U')
tarifa = tarifa * FACTOR_UBER;
contador = contador + 1;
Si no, Si (servicio == 'L')
tarifa = tarifa * FACTOR_LUJO;
contador = contador + 1;
Si (demanda == 'A')
tarifa = tarifa * FACTOR_DEMANDA_ALTA;
contador = contador + 1;
residuo = hora % 6;
contador = contador + 1;
Si no, Presentar: ("Error: Distancia, tiempo u hora invalidos.")
Presentar: ("Operacion realizada: Calculo de tarifa de transporte
privado")
Presentar: ("Tarifa total: " + tarifa)
Presentar: ("Residuo de division adicional (hora % 6): " + residuo)
Presentar: ("Operaciones realizadas: " + contador)
Fin
```

- ## Prueba de Escritorio

distancia	tiempo	servicio	demanda	horas	tarifa	residuo	contador
60	360	L	A	8	456	2	4
24	60	U	N	6	49.20	0	3

- **Codificación en C**

tarifa.c U

tarifa.c > ...

```c
#include <stdio.h>

int main() {
    double TARIFA_BASE = 2.0;
    double COSTO_KM = 1.0;
    double COSTO_MIN = 0.25;
    double FACTOR_UBER = 1.2;
    double FACTOR_LUJO = 2.0;
    double FACTOR_DEMANDA_ALTA = 1.5;
    double distancia, tiempo, tarifa = 0.0;
    char servicio = ' ';
    char demanda = ' ';
    int hora, residuo, contador = 0;

    printf("Ingrese la distancia en km: ");
    scanf("%lf", &distancia);
    printf("Ingrese el tiempo en minutos: ");
    scanf("%lf", &tiempo);
    printf("Ingrese el tipo de servicio ('T': taxi, 'U': uber, 'L': lujo): ");
    scanf(" %c", &servicio);
    printf("Ingrese la demanda ('B': baja, 'N': normal, 'A': alta): ");
    scanf(" %c", &demanda);
    printf("Ingrese la hora (0-23): ");
    scanf("%d", &hora);

    if ((distancia > 0.0) && (tiempo > 0.0) && (hora >= 0) && (hora <= 23)) {
        contador = contador + 1;
        tarifa = TARIFA_BASE + (distancia * COSTO_KM) + (tiempo * COSTO_MIN);

        if (servicio == 'U') {
            tarifa = tarifa * FACTOR_UBER;
            contador = contador + 1;
        } else {
            if (servicio == 'L') {
                tarifa = tarifa * FACTOR_LUJO;
                contador = contador + 1;
            }
        }

        if (demanda == 'A') {
            tarifa = tarifa * FACTOR_DEMANDA_ALTA;
            contador = contador + 1;
        }

        residuo = hora % 6;
        contador = contador + 1;

        printf("\nOperacion realizada: Calculo de tarifa de transporte privado\n");
        printf("Tarifa total: $%.2lf\n", tarifa);
        printf("Residuo de division adicional (hora %% 6): %d\n", residuo);
        printf("Operaciones realizadas registradas: %d\n", contador);
    }
    else {
        contador = contador + 1;
        printf("Error: La distancia, el tiempo o la hora estan fuera de los rangos validos.\n");
    }

    return 0;
}
```

Fig. 39. Código "tarifa.c"

- **Salida en la terminal:**

```
C:\java_project\primer-ciclo-2026_2027\codigosC>tarifa
Ingrese la distancia en km: 8
Ingrese el tiempo en minutos: 24
Ingrese el tipo de servicio ('T': taxi, 'U': uber, 'L': lujo): T
Ingrese la demanda ('B': baja, 'N': normal, 'A': alta): B
Ingrese la hora (0-23): 5

Operacion realizada: Calculo de tarifa de transporte privado
Tarifa total: $16.00
Residuo de division adicional (hora % 6): 5
Operaciones realizadas registradas: 2
```

Fig. 40. Compilación y ejecución.

- **Conclusiones:**
  1. Validar de forma estricta los datos de entrada (como las horas, que van entre 0 y 23) previene de fallos en el procesamiento y asegura resultados coherentes.
  2. Incluir una variable contadora permite llevar un registro exacto de cuantas reglas o pasos de cálculo se ejecutaron según las elecciones del usuario.

7. Anexos

Enlace de acceso al repositorio de GitHun:

https://github.com/noelia-quille/primer-ciclo-2026_2027